

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение
высшего образования
«УЛЬЯНОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ»

ФАКУЛЬТЕТ ИНФОРМАЦИОННЫХ СИСТЕМ И ТЕХНОЛОГИЙ

Кафедра «Информационные системы»

Дисциплина «Информационная безопасность»

Лабораторная работа № 4

Захват и анализ сетевого трафика

Выполнил:
студент гр. ПИБд-41
Белянин Н.Н.
Проверил:
Доцент, к.т.н.
Мытарев П. В.

Ульяновск, 2025г.

Лабораторная работа № 4

Захват и анализ сетевого трафика.

Целью данной работы является ознакомление с программой «The Wireshark Network analyzer», которая представляет собой программный анализатор протоколов. Программа содержит следующие основные компоненты: фильтр захвата, буфер кадров, декодер протоколов, фильтр отображения захваченных кадров и модуль статистики с элементами экспертной системы и обладает следующими возможностями:

- возможность захвата трафика в сетевых сегментах различных базовых технологий;
- возможность анализа большого числа протоколов;
- возможность экспорта и импорта файлов данных в формат распространенных анализаторов;
- мощная и удобная система поиска и фильтрации информации в буфере пакетов;
- наличие элементов экспертной системы;
- возможность сохранения на диск выделенного фрагмента пакета;
- наличие полезных утилит командной строки для осуществления захвата трафика и обработки сохраненных файлов.

После запуска программы Wireshark (Рис.1.) необходимо настроить некоторые параметры. Сначала необходимо открыть панель меню Capture => Options(Рис.2). На этой панели необходимо

установить следующие параметры захвата кадров:

- Interface - сетевой адаптер;
- Buffer size - размер буфера захвата (по умолчанию 1 Мб);
- Capture packets in promiscuous mode - использование режима беспорядочного захвата, если флажок снят, то захватываться будут кадры, предназначенные для указанного сетевого интерфейса;
- Limit each packet to - запись только нескольких первых байт (определяется установленным значением параметра) каждого кадра;
- Capture Filter - фильтр захвата, предназначен для фильтрации кадров при записи, увеличивает нагрузку на процессор;
- Capture File(s) - файл захвата;
- Stop Capture - условия автоматического завершения захвата;
- Display Options - отображение пакетов в реальном времени и автоматический скроллинг окна информации;
- Name Resolution - разрешение имен на физическом, сетевом и транспортном уровнях.

Результат выполнения захвата кадра при выполнении команды «ping mail.ru» приведен на рис.3. На экране монитора в программе присутствует несколько панелей с отображением сетевых пакетов, записанных в буфер. Пользовательский интерфейс программы содержит следующие компоненты:

- меню команд и панель инструментов;
- фильтр отображения пакетов;
- список пакетов в буфере;
- панель отображения декодера протоколов;
- панель отображения пакета в шестнадцатеричном коде и символах ASCII.

- Панель со списком пакетов построчно отображает характеристики того или иного пакета (номер по порядку в буфере, время захвата, адреса источника и получателя, тип протокола и общая информация о нем).

- В поле Filter используется для фильтрации захваченных данных. Используемыми переменными являются поля заголовков протоколов. Можно создавать довольно сложные условия с использованием операторов “!=”, “==”, “<”, “>”, “>=”, “<=”, а также OR, AND, NOT(!). Например, условие TCP.Flags.SYN == 1 означает отбор всех кадров, в которых протокол TCP осуществляет запрос на установление соединения.

- Поиск захваченных кадров можно осуществлять через меню Edit->Find Paket. Фильтр может содержать вышеописанные условия фильтрации, шестнадцатеричный код или данные типа «строка». Поиск может происходить в любой из трех областей – области пакетов, в области расшифровки протоколов пакета или непосредственно в пакете.

- В настройках программы можно отключать показ выбранных протоколов с помощью пункта Analyze->Enabled Protocols.

1. Перед выполнением лабораторной работы необходимо повторить механизмы работы и структуру стека протоколов TCP/IP(TCP,IP,ARP,ICMP), протоколы SMTP,POP3.
2. Запустите программу «The Wireshark Network analyzer».
3. Выберите необходимый сетевой интерфейс.
4. Снимите флажок опции «Capture packets in promiscuous mode».
5. Чтобы начать процесс захвата нажмите на кнопку «Start» (Рис.2) либо на соответствующий значок на панели управления.
6. В командной строке выполните команду ping на любой хост.
7. После завершения команды ping остановите захват, нажав кнопку «Stop».

8. Проанализируйте результаты работы команды, содержимое экрана.
9. Загрузите в программу файл POP3.рсар. В этом файле находятся результаты связи по протоколу POP3 с почтовым сервером mail.ru для учетной записи test0073. Проанализируйте содержимое захваченных пакетов, определите пароль учетной записи. Найдите все пакеты, содержащие имя учетной записи. Отфильтруйте с помощью опции меню сеанс связи из общего потока. Отфильтруйте команду ring на сервер pop3.mail.ru.
10. Загрузите в программу файл SMTP.рсар. В этом файле находятся результаты связи по протоколу SMTP с почтовым сервером mail.ru для учетной записи test0073. Отфильтруйте с помощью опции меню сеанс связи из общего потока.
11. Включите захват кадров, среди захваченных кадров с помощью фильтра оставьте только ICMP-сообщения. Для этого можно в поле фильтр набрать ICMP и применить этот фильтр. Если после набора icmp набрать символ «.», то откроется окно с возможными вариантами дальнейшего выбора параметров второго уровня.
12. Дайте команду ring на какой-либо узел. Установите фильтр для выбора протокола ARP, в поле IP-адрес назначения которого присутствовал бы набранный IP-адрес.
13. Отберите кадры, содержащие в поле «Протокол» IP-протокола протокол ICMP, где адресом назначения является выбранный Вами адрес в команде ring.
14. Найдите все пакеты по строке «request» в области расшифровки протоколов.
15. Захватите кадры при копировании файла размером не менее 10кБ с ftp-сервера. Отключите протоколы, находящиеся выше протокола TCP, оставьте только протокол TCP. Проанализируйте, как изменились области, как фрагментировалась дейтаграмма, сколько TCP-сегментов получилось и какого размера.
16. Захватите кадры при копировании файла 111.txt с ftp-сервера из директории с заданием для данной лабораторной работы. Найдите в захваченных кадрах переданную структуру каталога, имя передаваемого файла, его содержимое.
17. Составьте условия отбора, чтобы определить количество TCP-соединений в захваченных кадрах.
18. Изучите пункт меню Analyze.
19. Откройте статистику сеансов TCP, выберите первый сеанс из меню Apply as Filter -> Selected -> A<->B.

Оглавление

Ход выполнения лабораторной работы	6
Теоретическая часть выполнения.....	6
Анализатор трафика.....	7
Принцип работы sniffера.....	7
Применение анализаторов трафика	8
Структура сетевого пакета.....	9
Сетевая модель OSI	13
Сетевые протоколы.....	15
Протоколы информационной безопасности (ИБ)	18
Практическая часть выполнения	19
Wireshark.....	19
Основные возможности Wireshark.....	20
Работа с Wireshark.....	22
Демонстрация работы программной системы	25
Выводы.....	52
Заключение	55

Ход выполнения лабораторной работы

Теоретическая часть выполнения

Сегодня современные компьютерные сети представляют собой сложные распределённые системы, функционирование которых обеспечивается множеством сетевых протоколов. Эффективный анализ сетевого трафика является критически важной и нужной задачей для специалистов по информационной безопасности, системных администраторов, разработчиков сетевых приложений и всех, кто изучает принципы работы сетей. Анализ трафика позволяет диагностировать проблемы производительности, выявлять аномалии, атаки и утечки данных, оптимизировать конфигурацию сетевого оборудования и понимать логику взаимодействия между сетевыми узлами. В кибербезопасности анализ передачи и получения трафика позволяет выявить возможные уязвимости и дыры в безопасности, а также можно отслеживать сетевую активность пользователей.

Данная лабораторная работа посвящена освоению одного из нужных и широко используемых распространённых инструментов для анализа сетевого трафика – программы Wireshark Network Analyzer. Основной целью работы является приобретение практических навыков использования Wireshark для мониторинга, фильтрации и интерпретации сетевого трафика. В ходе выполнения работы будут решаться следующие задачи:

1. Изучение теоретической составляющей трафика данных, телекоммуникационных сетей.
2. Методы работы с телекоммуникационными сетями и специализированное программное обеспечение.
3. Ознакомление с интерфейсом и основными функциональными возможностями Wireshark.
4. Освоение методики захвата сетевых пакетов с настройкой ключевых параметров, таких как выбор сетевого интерфейса, режим захвата и применение фильтров захвата.

Данная лабораторная работа направлена на развитие практических навыков в анализе трафика. Все пункты заданий направлены на изучение возможностей анализатора Wireshark и приобретение опыта в области информационной безопасности, а теоретические знания помогут подробно разобраться и получить знания о структуре сетевых пакетов и кадров, о принципах работы протоколов и процедурах взаимодействия сетевых узлов.

Анализатор трафика

- **Анализатор трафика** или **сниффер** (досл.: «вынюхивающий» от англ. sniff — «нюхать») — это устройство и/или программное обеспечение, позволяющее считывать, анализировать, а в случае злонамеренности и искажать, сетевой трафик в месте собственного подключения к компьютерной сети.

Захват трафика (перехват трафика) в сетевых коммуникациях — это процесс захвата и анализа данных, которые проходят через сеть. Обычно это делается с целью мониторинга, диагностики или обеспечения безопасности сети.

Принцип работы сниффера

Сниффер может анализировать только то, что проходит через его сетевую карту. Внутри одного сегмента сети Ethernet, все пакеты рассылаются всем машинам, из-за этого возможно перехватывать чужую информацию. Для защиты от **прослушивания** используют коммутаторы с их грамотной конфигурацией. Информация между сегментами передаётся через коммутаторы.

Коммутация пакетов — это форма передачи, при которой данные, разбитые отдельные пакеты, могут пересылаться из исходного пункта в пункт назначения разными маршрутами. Так что если кто-то в другом

сегменте посылает внутри него какие-либо пакеты, то в ваш сегмент коммутатор эти данные не отправит.

Перехват трафика может осуществляться:

- Обычным «прослушиванием» сетевого интерфейса (метод эффективен при использовании в сегменте концентраторов (хабов) вместо коммутаторов (свитчей) в противном случае метод малоэффективен, поскольку на сниффер попадают лишь отдельные фреймы).
- Подключением сниффера в разрыв канала.
- Ответвлением (программным или аппаратным) трафика и направлением его копии на сниффер.
- Через анализ побочных электромагнитных излучений и восстановление, таким образом, прослушиваемого трафика.
- Через атаку на канальном (MAC) или сетевом уровне (IP), приводящую к перенаправлению трафика жертвы или всего трафика сегмента на сниффер с последующим возвращением трафика в надлежащий адрес.

Применение анализаторов трафика

В начале 1990-х широко применялся хакерами для захвата пользовательских логинов и паролей, которые в ряде сетевых протоколов передаются в незашифрованном или слабо зашифрованном виде. Широкое распространение хабов позволяло захватывать трафик без больших усилий в больших сегментах сети практически без риска быть обнаруженным.

Снифферы применяются как в деструктивных (разрушительный), так и в благих целях. Анализ прошедшего через сниффер трафика позволяет:

- Обнаружить вирусный и закольцованный трафик, наличие которого увеличивает загрузку сетевого оборудования и каналов связи

(снифферы малоэффективны, лучше для этого использовать сбор разнообразной статистики серверами и активным сетевым оборудованием и её последующий анализ).

- Выявить в сети вредоносное и несанкционированное программное обеспечение (сетевые сканеры, флудеры, троянские программы). Специализированные снифферы мониторят сетевую активность и обнаруживают её.

- Перехватить любой незашифрованный (а порой и зашифрованный) пользовательский трафик с целью получения паролей или другой информации.

- Локализовать неисправность сети или ошибку конфигурации сетевых агентов (для этой цели снифферы часто применяются системными администраторами)

Когда мы выполняем какие-либо действия в Интернете (посещение веб-сайтов, отправка писем или просто передаём интернет как роутер или маршрутизатор), данные разбиваются на более мелкие фрагменты или "пакеты" и отправляются по сети к месту назначения. Важным составляющим трафика является сетевой пакет.

Сетевой пакет (Packet) – это определённым образом оформленный блок данных, передаваемый по сети в пакетном режиме или отформатированная единица данных, передающаяся по сети.

Структура сетевого пакета

Сетевой пакет может состоять из следующих элементов:

- Служебная информация, которая включает в себя стартовые биты (преамбулу) некий двоичный код.
- Заголовки (headers) – адреса источника и назначения.

- Прицеп (trailer) – контрольная сумма (проверка) и стоповая комбинация.
- Полезная нагрузка (payload) – фактические данные или содержание сообщения.
- Метаданные (metadata) – информация о том, как обрабатывать пакет.

Разметка пакета

Пакет состоит из двух типов данных: *управляющей информации* и *данных пользователя* (также полезной нагрузкой). Управляющая информация содержит данные, необходимые для доставки данных пользователя: адреса отправителя и получателя, коды обнаружения ошибок (типа контрольных сумм) и информацию об очередности. Как правило, управляющая информация содержится в заголовке и хвосте пакета, а между ними размещаются пользовательские данные.

Сетевой пакет: структура

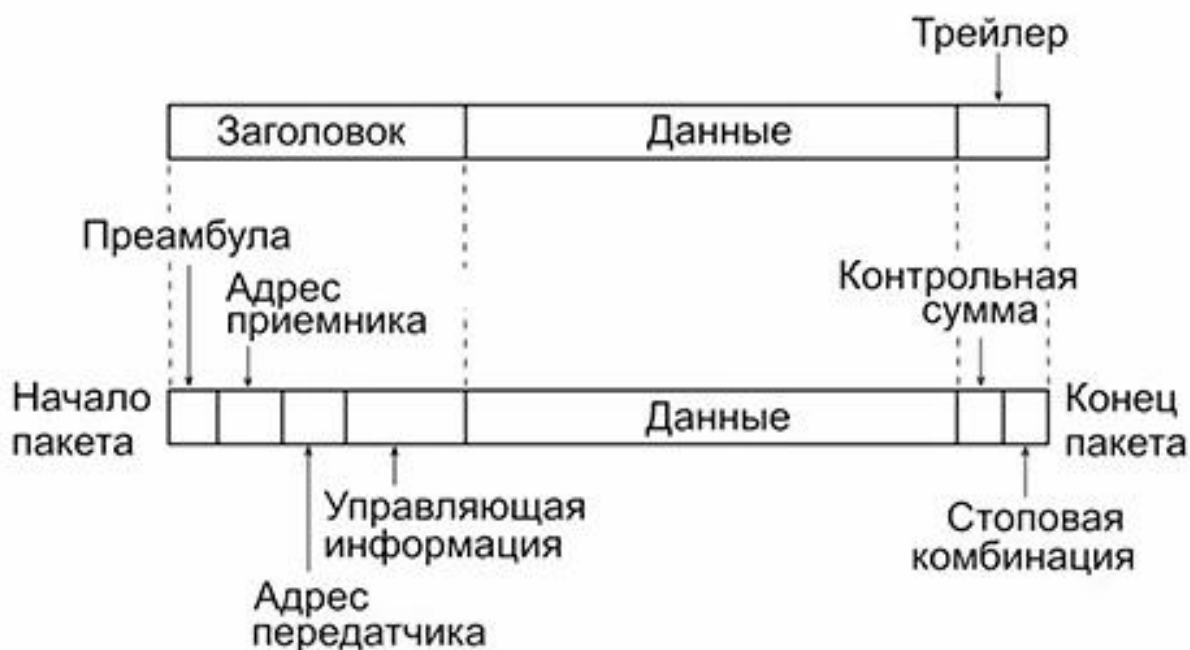


Рисунок 1. Структура сетевого пакета.

Точная структура пакета задаётся протоколом. Пакеты формируются на канальном, сетевом, транспортном уровне и иногда на прикладном.

Между пакетами, посылаемыми в сеть, обычно соблюдается межкадровый интервал (Interframe gap). Максимальная длина нагрузки называется maximum transmission unit (MTU).

Существует возможность *фрагментации пакета* – генерация двух сетевых пакетов из одного. Происходит это из-за превышения длины кадра MTU интерфейса, через который он в данный момент проходит. Фрагментация поддерживается протоколом IP и не предусмотрена в большинстве других протоколов. Если сетевой адаптер обнаруживает кадр длиннее его MTU, то этот кадр обычно отбрасывается. Фрагментация IP-пакета увеличивает нагрузку на центральный процессор и снижает скорость передачи полезных данных этого пакета. При потере любого фрагмента, необходимо повторно передать всю последовательность, а это является дополнительным риском снижения скорости. Сборка всех частей в исходный пакет производится только адресатом, даже если на каком-то участке сети MTU больше требуемого.

Компьютерные линии связи, которые не поддерживают пакетного режима, как традиционная телекоммуникационная связь *"точка-точка"* передают данные просто в виде последовательности *байтов, символов или битов* поодиночке. Если данные сформированы в пакеты, *битрейт* коммуникационной среды можно более эффективно распределить между пользователями, чем в сети с коммутацией каналов. При использовании сетей с коммутацией пакетов можно надёжно гарантировать пороговый битрейт, ниже которого он опускаться не будет.

Различные коммуникационные протоколы используют разные соглашения для разделения элементов и для форматирования данных. В

протоколе «двоичной синхронной передачи» пакет отформатирован в 8-битных байтах, а для разделения элементов используются специальные символы. В других протоколах, таких как Ethernet, зафиксировано начало заголовка и элементов данных, их расположение относительно начала пакета. Некоторые протоколы форматируют информацию на уровне битов, а не байтов.

Применение пакетов: в кибербезопасности понимание структуры пакета имеет основополагающее значение для обнаружения подозрительного поведения (например, пакет с необычными адресами источника или портами назначения может указывать на чью-то попытку сканирования вашей сети).

При проектировании сети с применением пакетов можно достичь двух важных результатов: *обнаружение ошибок* и *многохостовая адресация*.

- **Обнаружение ошибок.** Более эффективным и надёжным методом обнаружения ошибок является расчёт контрольной суммы или циклического избыточного кода над содержимым пакета, чем проверка каждого символа с помощью бита чётности. Хвостовая часть пакета часто содержит данные проверки ошибок, возникших во время передачи пакета по сети.

- **Адрес хоста.** Современные сети обычно соединяют между собой три или более хоста. В таких случаях заголовок пакета обычно содержит информацию, в которой записан фактический адрес хоста. В сложных сетях, построенных из нескольких узлов коммутации и маршрутизации, такие как ARPANET или современный интернет, ряд пакетов, отправленных с одного компьютера на другой, может следовать разными маршрутами. Эта технология называется *пакетной коммутацией*.

Сетевая модель OSI

- **Сетевая модель OSI** (The Open Systems Interconnection model) – это сетевая модель стека (магазина) сетевых протоколов OSI/ISO. Посредством данной модели различные сетевые устройства могут взаимодействовать друг с другом. Модель определяет различные уровни взаимодействия систем. Каждый уровень выполняет определённые функции при таком взаимодействии.

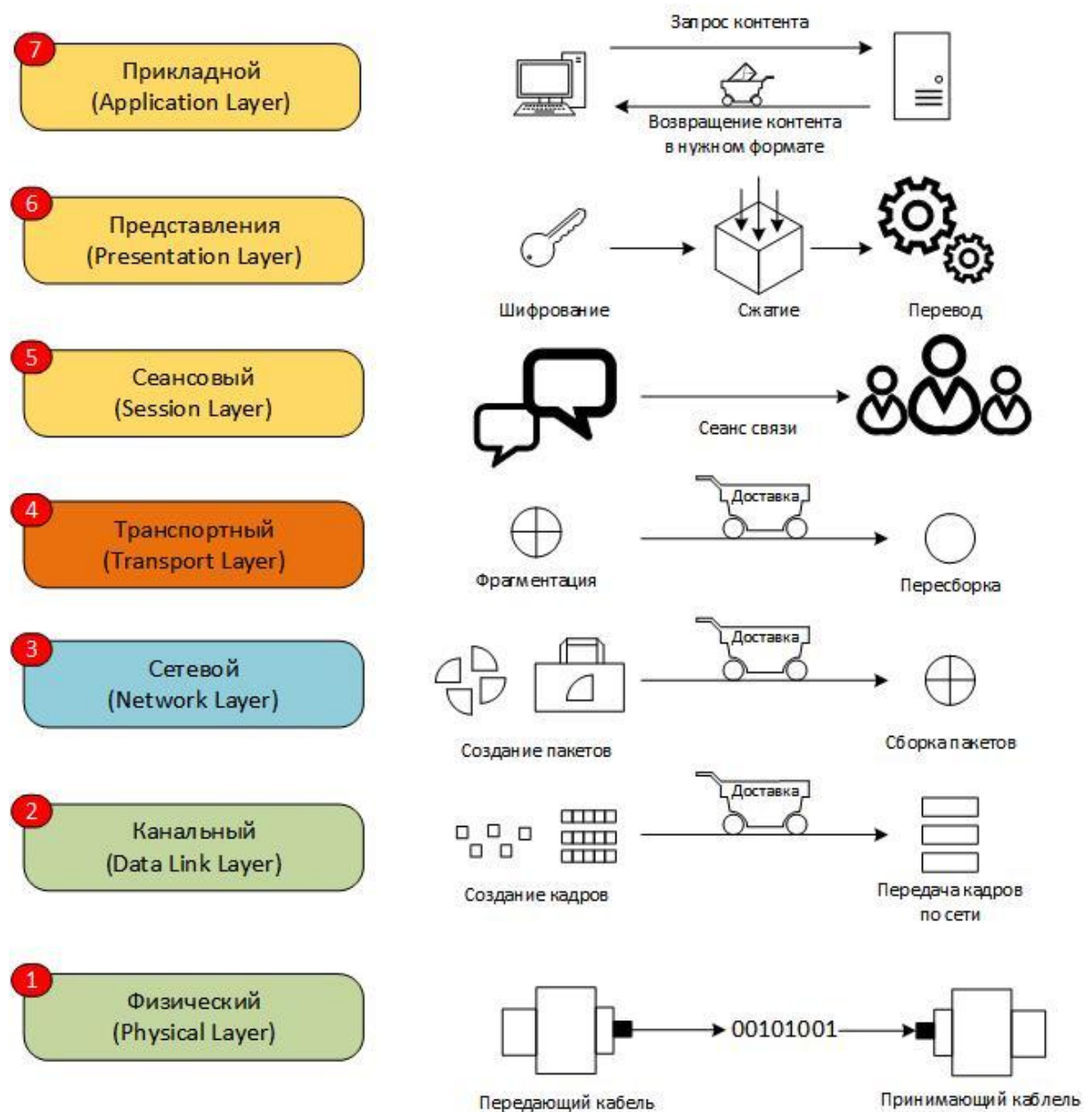


Рисунок 2. Модель OSI.

Модель открытых систем OSI					
Уровень (layer)		Тип данных	Функции уровня	Особенность адресации	Примеры протоколов
Уровни хоста (узла)	7. Прикладной (application)	Данные (строки из байтов)	Доступ к сетевым службам	URL	HTTP(S), FTP(S), RPC, POP3
	6. Представительский (представления) (presentation)		Представление (кодировка) и шифрование данных		ASCII, EBCDIC
	5. Сеансовый (session)		Управление сеансом связи		PAP
	4. Транспортный (transport)	Сегменты (segment) / Дейтаграммы (datagram)	Прямая связь между конечными пунктами и надёжность	Порт	TCP, UDP, SCTP, PORTS
Уровни связи (сети)	3. Сетевой (network)	Пакеты (packet)	Определение маршрута и логическая адресация	IP-адрес	IPv4, IPv6, IPsec, AppleTalk
	2. Канальный (data link)	Биты (bit) / Кадры (frame)	Физическая адресация	MAC-адрес (физический адрес компьютера)	PPP, IEEE 802.22, Ethernet, DSL, ARP, L2TP, сетевая карта.
	1. Физический (physical)	Электрические сигналы, Биты (bit)	Работа со средой передачи, сигналами и двоичными данными		USB, кабель ("витая пара", коаксиальный, оптоволоконный), радиоканал

Рисунок 3. Модель открытых систем OSI (таблицей).

Семейство TCP/IP

TCP/IP — это сетевая модель передачи данных, представленных в цифровом виде. Модель описывает способ передачи данных от источника информации к получателю. В модели предполагается прохождение информации через четыре уровня, каждый из которых описывается правилом (протоколом передачи). Наборы правил, решающих задачу по передаче данных, составляют стек протоколов передачи данных, на которых базируется Интернет. Название TCP/IP происходит из двух важнейших протоколов семейства — Transmission Control Protocol (TCP) и Internet Protocol (IP), которые были первыми разработаны и описаны в данном стандарте. Также изредка упоминается как модель DOD (Department of Defense) в связи с историческим происхождением от сети ARPANET из 1970-х годов (под управлением DARPA, Министерства обороны США).

Семейство TCP/IP имеет три транспортных протокола: TCP, полностью соответствующий OSI, обеспечивающий проверку получения данных; UDP, отвечающий транспортному уровню только наличием порта,

обеспечивающий обмен датаграммами между приложениями, не гарантирующий получения данных; и SCTP, разработанный для устранения некоторых недостатков TCP, в который добавлены некоторые новшества. В семействе TCP/IP есть ещё около двухсот протоколов, самым известным из которых является служебный протокол ICMP, используемый для внутренних нужд обеспечения работы; остальные также не являются транспортными протоколами.

Семейство IPX/SPX

В семействе IPX/SPX порты появляются в протоколе сетевого уровня IPX, обеспечивая обмен датаграммами между приложениями (операционная система резервирует часть сокетов для себя). Протокол SPX, в свою очередь, дополняет IPX всеми остальными возможностями транспортного уровня в полном соответствии с OSI. В качестве адреса хоста IPX использует идентификатор, образованный из четырёхбайтного номера сети (назначаемого маршрутизаторами) и MAC-адреса сетевого адаптера.

Сетевые протоколы

При работе с телекоммуникационными сетями важно рассмотреть сетевые протоколы их назначение

Набор интернет-протоколов обеспечивает сквозную передачу данных, определяющую, как данные должны пакетироваться, обрабатываться, передаваться, маршрутизироваться и приниматься. Эта функциональность организована в слои абстракции, которые классифицируют все связанные протоколы в соответствии с объёмом задействованных сетей. От самого низкого до самого высокого уровня:

- *Физический уровень (L1)* – отвечает за передачу битов по физической среде. Протоколы: не применяются напрямую – используется спецификация (Ethernet, Wi-Fi PHY, DSL).

- *Канальный уровень (L2)* — это уровень связи, содержащий методы связи для данных, которые остаются в пределах одного сегмента сети. Обеспечивает передачу данных внутри локальной сети.

Протоколы:

- ✓ Ethernet – стандарт LAN.
- ✓ Wi-Fi (IEEE 802.11) – беспроводной LAN.
- ✓ PPP (Point-to-Point Protocol) – соединение "точка-точка".
- ✓ ARP (Address Resolution Protocol) – преобразует IP → MAC.
- ✓ RARP / InARP – обратные ARP (устаревшие).
- ✓ STP (Spanning Tree Protocol) – предотвращает петли в сетях.
- ✓ LLDP (Link Layer Discovery Protocol) – обнаружение соседних устройств.

- *Сетевой уровень (L3)* – обеспечивает межсетевое взаимодействие между независимыми сетями. Роутинг и адресация между сетями.

Протоколы:

- ✓ IP (Internet Protocol) – IPv4/IPv6.
- ✓ ICMP (Internet Control Message Protocol) – диагностика (ping, traceroute).
- ✓ IGMP (Internet Group Management Protocol) – управление multicast-группами.
- ✓ IPsec (часть L3, но также L4) – шифрование и аутентификация IP-пакетов.
- ✓ GRE / L2TP / PPTP – туннелирование (часто используется в VPN).

- *Транспортный уровень (L4)* — обрабатывает связь между хостами. Надёжная или ненадёжная доставка данных между приложениями.

Протоколы:

- ✓ TCP (Transmission Control Protocol) – надёжная доставка с подтверждениями.

- ✓ UDP (User Datagram Protocol) – ненадёжная, но быстрая доставка.
- ✓ SCTP (Stream Control Transmission Protocol) – альтернатива TCP/UDP.
- ✓ DCCP – для мультимедийных приложений.

- *Сеансовый уровень (L5)* – управление сессиями и сеансами между приложениями.

Протоколы:

- ✓ TLS/SSL – часто относят сюда или к L6/L7.
- ✓ NetBIOS – устаревший протокол для Windows.
- ✓ RPC (Remote Procedure Call) – вызов процедур на удалённых машинах.

- *Уровень представления (L6)* – формат данных, шифрование, сжатие.

Протоколы:

- ✓ SSL/TLS – шифрование и целостность.
- ✓ MIME – формат передачи вложений.
- ✓ XDR (External Data Representation) – сериализация данных.

- *Прикладной уровень (L7)* — обеспечивает обмен данными между процессами для приложений. Протоколы, с которыми взаимодействует пользователь и приложение.

Протоколы:

- ✓ HTTP/HTTPS – веб-коммуникации.
- ✓ FTP/FTPS/SFTP – передача файлов.
- ✓ SMTP/POP3/IMAP – электронная почта.
- ✓ DNS – преобразование имён в IP.
- ✓ DHCP – автоматическая настройка IP.
- ✓ SNMP – управление сетевыми устройствами.

- ✓LDAP – доступ к каталогам.
- ✓NTP – синхронизация времени.
- ✓SIP / RTP – VoIP и мультимедиа.
- ✓MQTT / CoAP – IoT-протоколы.

Развитием архитектуры Интернета и протоколов в модели TCP/IP занимается открытое международное сообщество проектировщиков IETF.

TCP (англ. Transmission Control Protocol — протокол управления передачей) — один из основных протоколов передачи данных интернета. Пакеты в TCP называются сегментами. В стеке протоколов TCP/IP выполняет функции транспортного уровня модели OSI. Механизм TCP предоставляет поток данных с предварительной установкой соединения, осуществляет повторный запрос данных в случае потери данных и устраняет дублирование при получении двух копий одного пакета, гарантируя тем самым (в отличие от UDP) целостность передаваемых данных и уведомление отправителя о результатах передачи.

Реализации TCP обычно встроены в ядра ОС. Существуют реализации TCP, работающие в пространстве пользователя.

Протоколы информационной безопасности (ИБ)

Эти протоколы обеспечивают *доступность*, *целостность*, *конфиденциальность*, а так же аутентификацию и неотказуемость.

1. Криптографические протоколы

- ✓TLS / SSL – шифрование трафика (HTTPS, SMTPS и др.).
- ✓IPsec – защищает IP-трафик на сетевом уровне.
- ✓SSH (Secure Shell) – безопасный удалённый доступ к системам.
- ✓PGP / S/MIME – шифрование и подпись электронной почты.
- ✓Kerberos – сетевая аутентификация с использованием билетов.

✓ **OAuth 2.0 / OpenID Connect** – делегированная аутентификация и авторизация.

✓ **SAML** – федеративная идентификация.

2. Протоколы управления доступом и аутентификации

✓ **RADIUS** – аутентификация удалённых пользователей.

✓ **TACACS+** – более гибкий и безопасный, чем RADIUS.

✓ **LDAP / LDAPS** – доступ к каталогам пользователей.

✓ **802.1X** – аутентификация устройств в LAN/WLAN.

3. Протоколы мониторинга и защиты

✓ **SNMPv3** – защищённый мониторинг.

✓ **Syslog over TLS** – безопасный сбор логов.

✓ **NetFlow / sFlow** – анализ сетевого трафика.

✓ **SIEM-протоколы** – CEF, LEEF, JSON logs.

4. Протоколы обнаружения и предотвращения вторжений

✓ **IDS/IPS** используют стандартные протоколы, но сами по себе не протоколы, а системы, анализирующие трафик по протокольным сигнатурам.

Практическая часть выполнения

Wireshark

Wireshark – это один из самых широко используемых инструментов для анализа сетевого трафика. Он является ключевым инструментарием как для системных администраторов и специалистов по информационной безопасности, так и для хакеров (как этичных, так и неэтичных). С помощью него можно детально изучать трафик, проходящий через сеть, что делает его полезным как для диагностики проблем, так и для поиска уязвимостей.

Wireshark – это бесплатный и открытый инструмент (open-source) для анализа сетевых пакетов. Он работает как сниффер (sniffer), позволяя захватывать и декодировать сетевые пакеты, которые проходят через сетевой интерфейс компьютера. Благодаря этому, Wireshark предоставляет возможность изучать структуру сетевого трафика на уровне протоколов. Изначально Wireshark разрабатывался для диагностики сетевых проблем, но его возможности позволяют использовать его для более сложных задач, включая анализ безопасности.

История программы началась в 1997 году, когда Джеральд Комбс задумал создать инструмент для детального изучения сетевого трафика. Уже в 1998 году вышла первая версия под названием Ethereal, но позже проект был переименован в Wireshark. С активной поддержкой сообщества разработчиков инструмент быстро развивался, и в 2008 году вышел его первый стабильный релиз под номером 1.0. В 2025 году Wireshark остаётся полезным инструментом и анализатором сетевых протоколов в мире.

Основные возможности Wireshark

1. *Захват сетевого трафика.* Wireshark может захватывать пакеты в реальном времени или анализировать уже записанные данные из файлов сессий (например, формата .pcap). Захват возможен на различных уровнях сетевой модели OSI, включая Ethernet, TCP, UDP, HTTP и многие другие.

2. *Декодирование протоколов.* Wireshark поддерживает огромный список сетевых протоколов (более 1000), включая HTTP, TLS, FTP, DNS, ICMP, ARP. Это позволяет анализировать содержимое пакетов, включая заголовки и полезную нагрузку.

3. *Фильтрация данных.* Используя фильтры (Display Filters), можно выделять только интересующий трафик, например, пакеты от определенного

IP-адреса, по протоколу или портам. Это упрощает анализ, если объем захваченного трафика велик.

4. *Диагностика и анализ.* С помощью Wireshark можно выявлять сетевые проблемы, такие как потеря пакетов, высокие задержки, некорректная настройка маршрутизации. Также можно искать подозрительный трафик, возможные атаки (например, ARP-spoofing или попытки взлома через brute force).

5. *Экспорт данных.* Результаты анализа можно сохранять в различных форматах для дальнейшего использования или отчетности.

Как Wireshark используется хакерами?

Wireshark, как инструмент анализа трафика, может быть использован хакерами для следующих целей:

1. *Перехват данных (Сниффинг).* В незашифрованных протоколах (например, HTTP или FTP) можно извлечь такие данные, как логины, пароли, содержимое сообщений и файлы (сейчас чаще используется HTTPS чем HTTP (по понятным причинам), но все же HTTP тоже используется). В локальных сетях хакеры могут использовать Wireshark в сочетании с техниками атаки ARP-spoofing для перенаправления трафика жертвы через свой компьютер.

2. *Анализ и изучение сетевой инфраструктуры.* Хакеры используют Wireshark для изучения структуры сети, активных устройств, используемых протоколов и портов. Это помогает в планировании атак, например, для поиска уязвимых сервисов.

3. *Обход защитных механизмов.* Анализ сетевого трафика позволяет хакерам понять, как работают межсетевые экраны, системы обнаружения атак (IDS) и другие защитные механизмы.

4. *Поиск уязвимостей.* Wireshark помогает выявить неправильно настроенные системы, утечки данных или другие уязвимости, которые можно использовать для дальнейших атак.

Работа с Wireshark

Для начала необходимо скачать и установить Wireshark, по умолчанию всё. Найти установщик можно на официальном сайте <https://www.wireshark.org/>

Дальнейшая установка программы довольно проста, ознакомливается с соглашениями и следуем инструкциям установки.

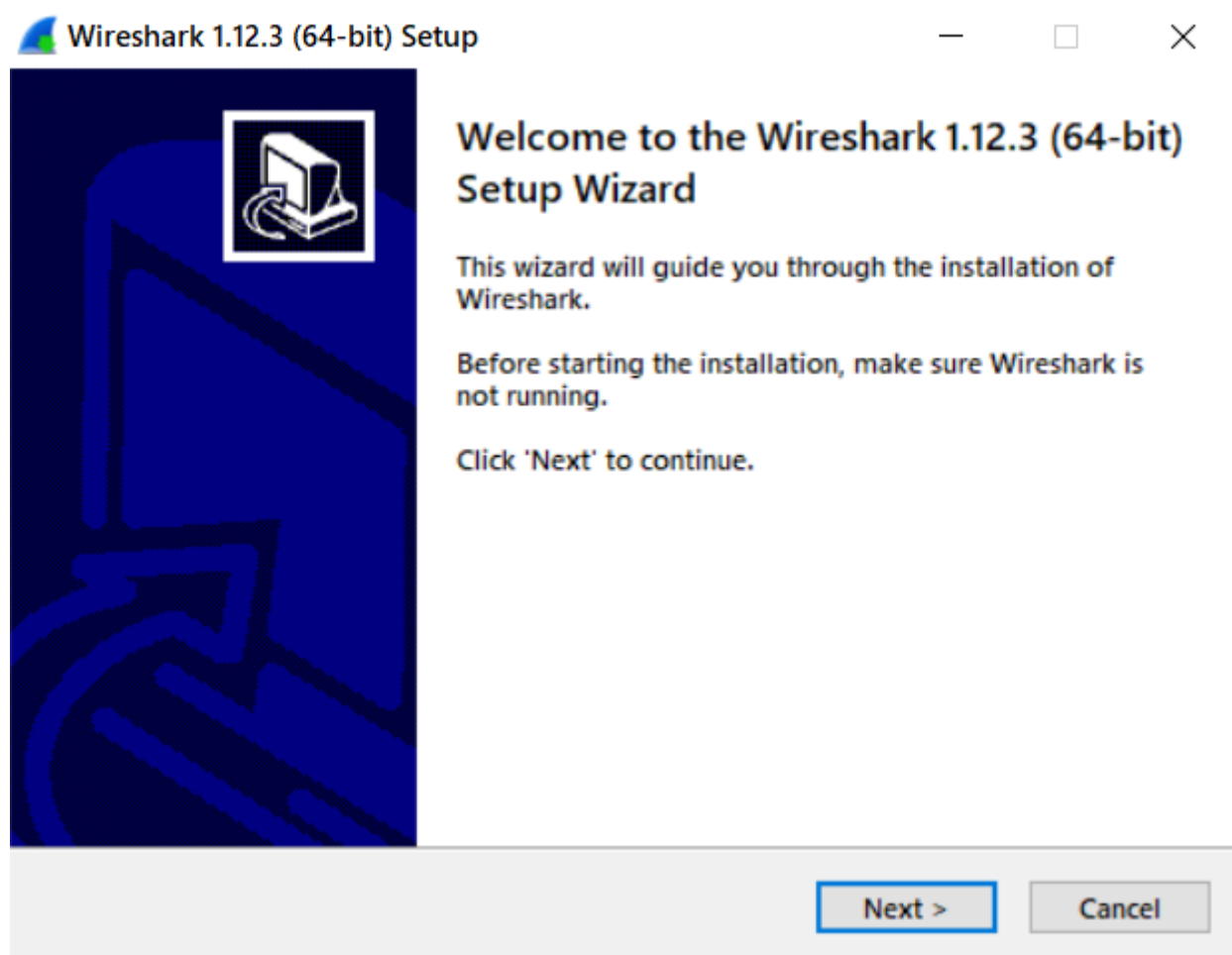


Рисунок 4. Установка Wireshark.

После успешной установки на рабочем столе появится ярлык Wireshark. Далее нужно перезагрузиться и запустить программу. Нас

встретит стартовое окно и меню, на котором покажут доступные интерфейсы компьютера. Мы будем производить захват пакетов через сетевой интерфейс Ethernet.

Сетевой интерфейс – это программное обеспечение, которое взаимодействует с сетевым драйвером и с уровнем IP. Он обеспечивает уровню IP доступ ко всем имеющимся сетевым адаптерам, трафик которых мы будем перехватывать. Чаще всего в программе Wireshark можно встретить сетевой интерфейс беспроводной (Wi-Fi) и кабельный (Ethernet).

Выбор сетевого интерфейса

Прежде чем захватывать пакеты, вам нужно выбрать, какой сетевой интерфейс (network interface) отслеживать. Каждый интерфейс представляет собой различное сетевое соединение в вашей системе. В главном окне Wireshark вы увидите список доступных сетевых интерфейсов. Рядом с каждым интерфейсом находится небольшой график, показывающий текущую активность.

Дважды щелкаем интерфейс eth1. Обычно это основной интерфейс Ethernet. Если вы не видите eth1, выберите любой интерфейс, который показывает активность (движущийся график).

Совет: График рядом с каждым интерфейсом показывает уровень сетевой активности на этом интерфейсе. Выбор интерфейса с большей активностью даст более интересные результаты захвата.

После того, как вы выберете интерфейс, Wireshark должен автоматически начать захват пакетов. Если этого не произошло, вы можете запустить захват вручную, щелкнув значок синего акульего плавника на панели инструментов или нажав Ctrl+E.

Во время захвата обратите внимание на основной список пакетов в окне Wireshark. Каждая строка представляет собой один пакет с несколькими столбцами информации:

No.: Номер пакета в этой сессии захвата

Time: Время захвата пакета

Source: IP-адрес или имя хоста отправляющего устройства

Destination: IP-адрес или имя хоста принимающего устройства

Protocol: Используемый сетевой протокол (TCP, UDP, HTTP и т. д.)

Length: Размер пакета в байтах

Info: Дополнительные сведения о пакете

Остановка захвата

После захвата в течение 30-60 секунд остановите захват, щелкнув значок красного квадрата на панели инструментов или снова нажав Ctrl+E.

Понимание панели деталей пакета

Панель деталей пакета показывает многоуровневую информацию о пакете, отражающую стек сетевых протоколов (network protocol stack):

1. **Уровень кадра (Frame Layer):** Физическая информация о захвате пакета (время захвата, длина кадра)
2. **Уровень Ethernet:** Содержит MAC-адрес и другую информацию канального уровня (link layer)
3. **Уровень IP (Internet Protocol):** Содержит информацию об IP-адресе
4. **Транспортный уровень (TCP/UDP):** Содержит номера портов и информацию о сессии
5. **Прикладной уровень (HTTP, DNS):** Содержит данные, специфичные для приложения

Общие знания: Это многоуровневое отражение соответствует модели OSI (Open Systems Interconnection) или модели TCP/IP, которая является

стандартной моделью для сетевой коммуникации. Понимание этих уровней помогает вам понять, как работает сетевая коммуникация.

Использование инструментов навигации Wireshark

Wireshark предоставляет несколько полезных инструментов навигации, которые помогут вам просматривать захват:

- **Панель инструментов фильтра (Filter Toolbar):** Расположена в верхней части главного окна, позволяет фильтровать отображаемые пакеты
- **Инструменты масштабирования (Zoom Tools):** Позволяют увеличивать или уменьшать масштаб представления оси времени
- **Цветовое кодирование (Color Coding):** Разные типы пакетов используют разные цвета, что упрощает визуальную идентификацию
- **Функция поиска (Find Feature) (Ctrl+F):** Позволяет искать конкретную информацию в содержимом пакета

Демонстрация работы программной системы

Приступим к работе над программой Wireshark. Перед работой зайдём в параметры в раздел "Захват" и отключим "Захват в смешанном режиме"

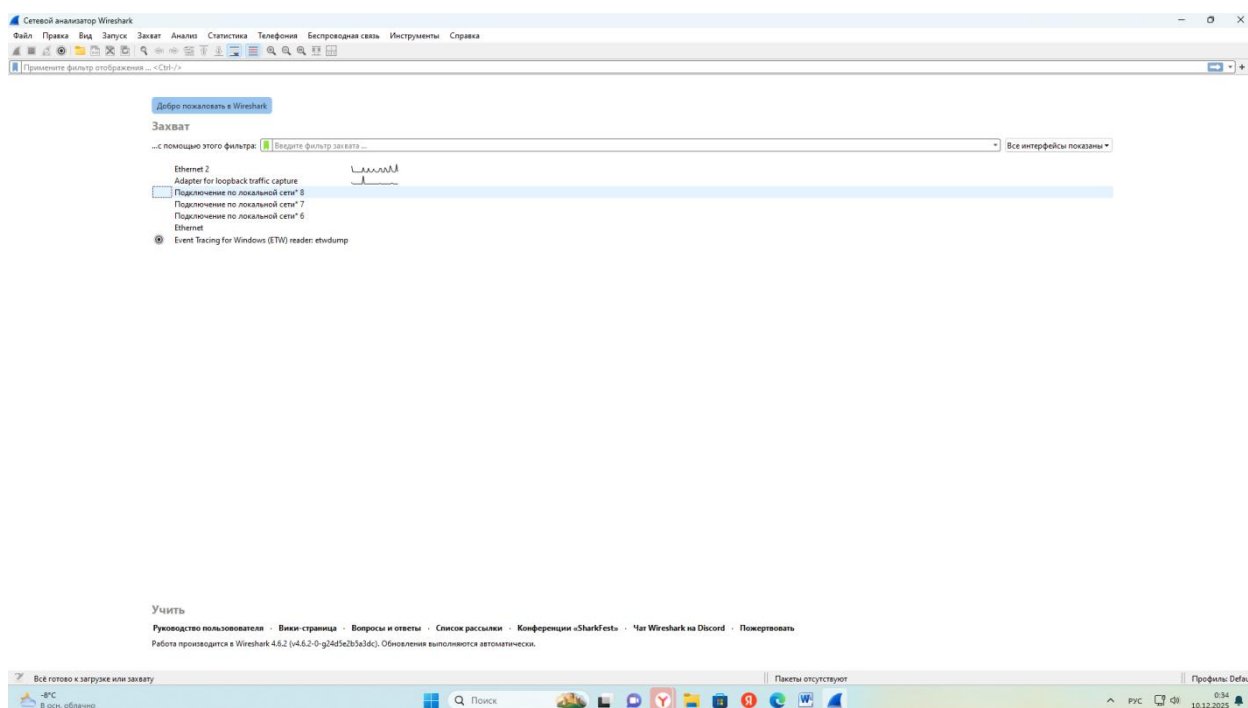


Рисунок 5. Главное меню Wireshark.

Переходим в раздел параметров для отключения надстройки.

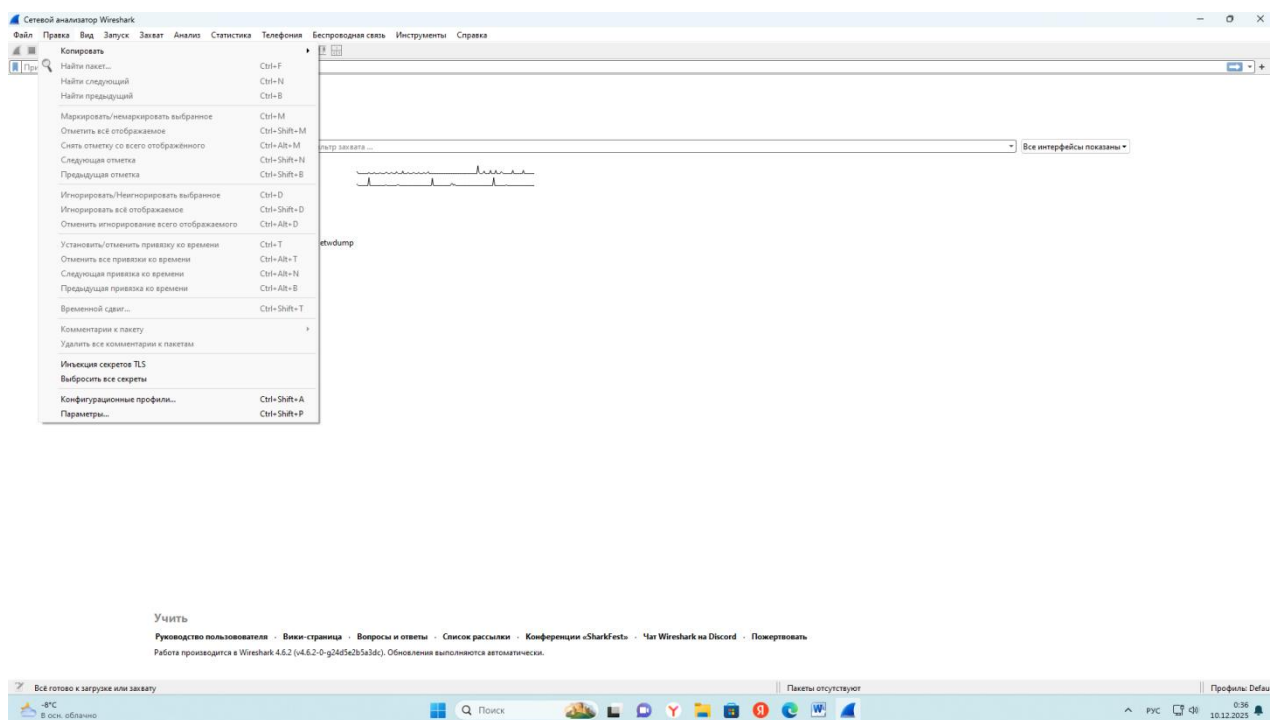


Рисунок 6. Раздел с параметрами Wireshark.

Отключаем смешанный режим при захвате пакетов.

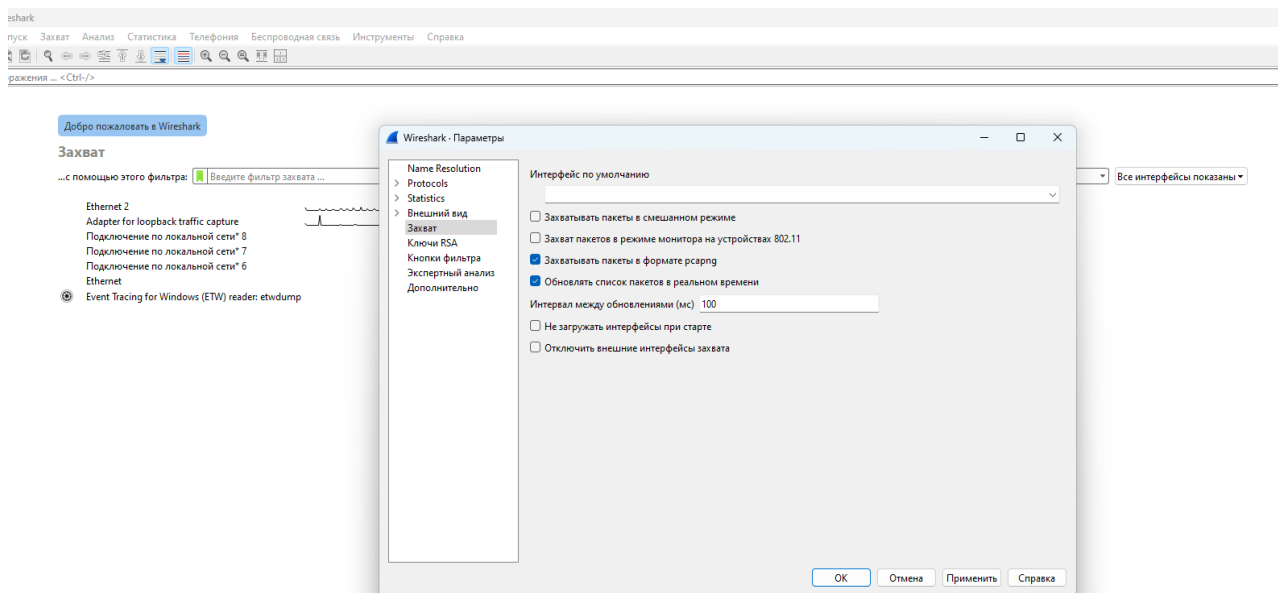


Рисунок 7. Отключаем смешанный режим захвата пакетов.

Далее с помощью команды **ping** пропингуем сайт Google.com и адрес публичного **DNS-сервера** сервиса Google Public DNS. Это бесплатный

глобальный сервис системы доменных имён (DNS), разработанный компанией Google.

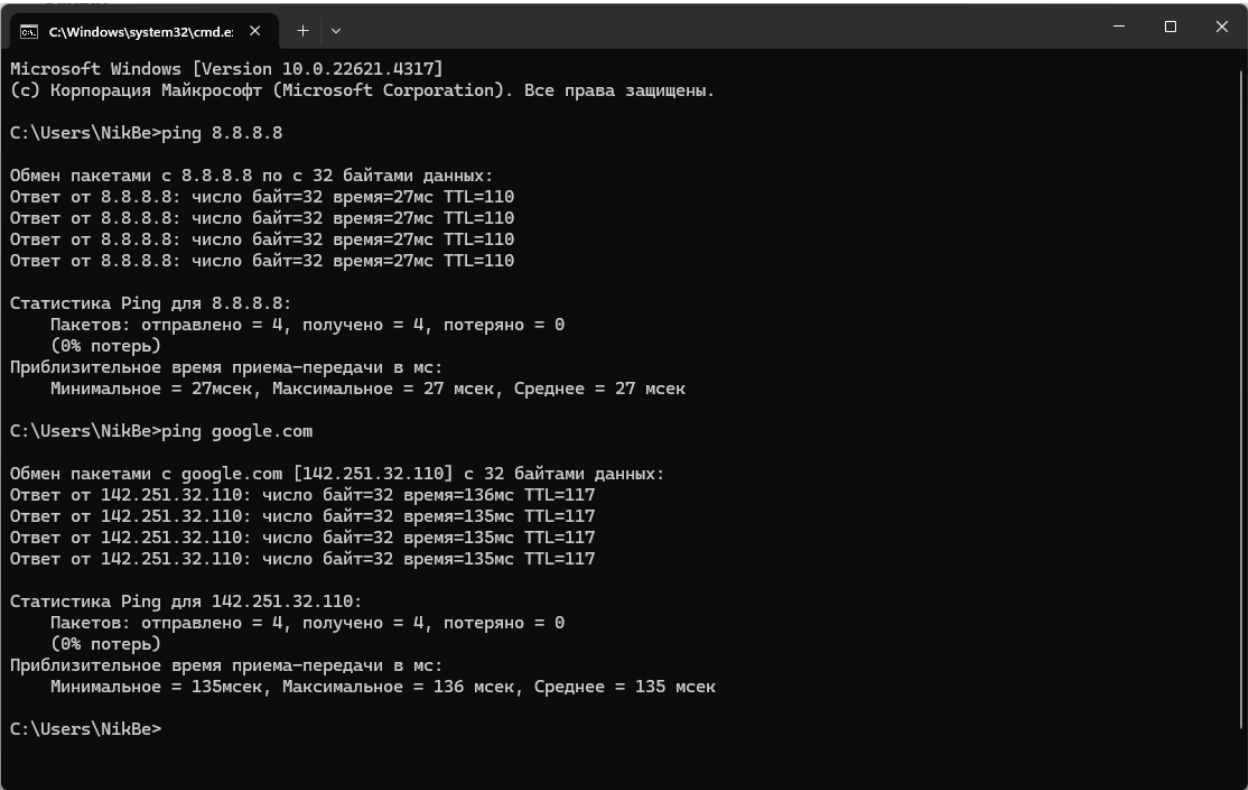


Рисунок 8. Выполнение обращения к сервисам Google с получением пакетов.

В Wireshark отображаются ответы от сервисов по соответствующим адресам и видны обращения к dns и к самому сайту.

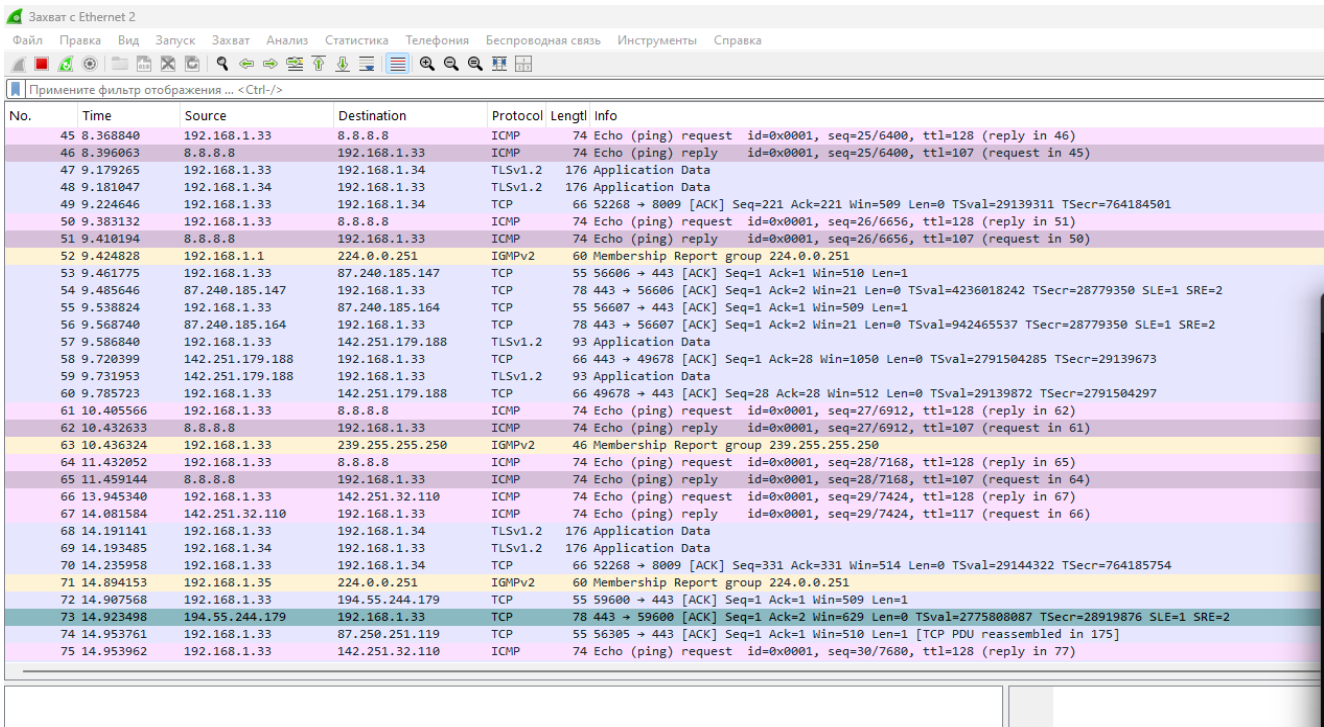


Рисунок 9. Ответ от dns сервера google.

Захват с Ethernet 2

Файл Правка Вид Запуск Захват Анализ Статистика Телефония Беспроводная связь Инструменты Справка

... <Ctrl-/>

No.	Time	Source	Destination	Protocol	Length	Info
63	10.436324	192.168.1.33	239.255.255.250	IGMPv2	46	Membership Report group 239.255.255.250
64	11.432052	192.168.1.33	8.8.8.8	ICMP	74	Echo (ping) request id=0x0001, seq=28/7168, ttl=128 (reply in 65)
65	11.459144	8.8.8.8	192.168.1.33	ICMP	74	Echo (ping) reply id=0x0001, seq=28/7168, ttl=107 (request in 64)
66	13.945340	192.168.1.33	142.251.32.110	ICMP	74	Echo (ping) request id=0x0001, seq=29/7424, ttl=128 (reply in 67)
67	14.081584	142.251.32.110	192.168.1.33	ICMP	74	Echo (ping) reply id=0x0001, seq=29/7424, ttl=117 (request in 66)
68	14.191141	192.168.1.33	192.168.1.34	TLSv1.2	176	Application Data
69	14.193485	192.168.1.34	192.168.1.33	TLSv1.2	176	Application Data
70	14.235958	192.168.1.33	192.168.1.34	TCP	66	52268 → 8009 [ACK] Seq=331 Ack=331 Win=514 Len=0 TSval=29144322 TSecr=764185754
71	14.894153	192.168.1.35	224.0.0.251	IGMPv2	60	Membership Report group 224.0.0.251
72	14.907568	192.168.1.33	194.55.244.179	TCP	55	59600 → 443 [ACK] Seq=1 Ack=1 Win=509 Len=1
73	14.923498	194.55.244.179	192.168.1.33	TCP	78	443 → 59600 [ACK] Seq=1 Ack=2 Win=629 Len=0 TSval=2775808087 TSecr=28919876 SLE=1 SRE=2
74	14.953761	192.168.1.33	87.250.251.119	TCP	55	5395 → 443 [ACK] Seq=1 Ack=1 Win=510 Len=1 [TCP PDU reassembled in 173]
75	14.953962	192.168.1.33	142.251.32.110	ICMP	74	Echo (ping) request id=0x0001, seq=30/7680, ttl=128 (reply in 77)
76	14.975761	87.250.251.119	192.168.1.33	TCP	78	443 → 56385 [ACK] Seq=1 Ack=2 Win=166 Len=0 TSval=3413964734 TSecr=29100013 SLE=1 SRE=2
77	15.089693	142.251.32.110	192.168.1.33	ICMP	74	Echo (ping) reply id=0x0001, seq=30/7680, ttl=117 (request in 75)
78	15.975867	192.168.1.33	142.251.32.110	ICMP	74	Echo (ping) request id=0x0001, seq=31/7936, ttl=128 (reply in 79)
79	16.111590	142.251.32.110	192.168.1.33	ICMP	74	Echo (ping) reply id=0x0001, seq=31/7936, ttl=117 (request in 78)
80	16.291743	192.168.1.33	142.251.179.188	TLSv1.2	118	Application Data
81	16.430509	142.251.179.188	192.168.1.33	TCP	66	443 → 49678 [ACK] Seq=28 Ack=80 Win=1050 Len=0 TSval=2791510995 TSecr=29146378
82	16.990171	192.168.1.33	142.251.32.110	ICMP	74	Echo (ping) request id=0x0001, seq=32/8192, ttl=128 (reply in 83)
83	17.125772	142.251.32.110	192.168.1.33	ICMP	74	Echo (ping) reply id=0x0001, seq=32/8192, ttl=117 (request in 82)
84	17.503542	104.18.94.41	192.168.1.33	TLSv1.2	105	Application Data
85	17.503542	104.18.94.41	192.168.1.33	TCP	66	443 → 56498 [FIN, ACK] Seq=40 Ack=1 Win=17 Len=0 TSval=2682272521 TSecr=28747568
86	17.503896	192.168.1.33	104.18.94.41	TCP	66	56498 → 443 [ACK] Seq=1 Ack=41 Win=514 Len=0 TSval=29147590 TSecr=2682272521
87	17.504334	192.168.1.33	104.18.94.41	TCP	66	56498 → 443 [FIN, ACK] Seq=1 Ack=41 Win=514 Len=0 TSval=29147590 TSecr=2682272521
88	17.812077	192.168.1.33	104.18.94.41	TCP	66	[TCP Retransmission] 56498 → 443 [FIN, ACK] Seq=1 Ack=41 Win=514 Len=0 TSval=29147898 TSecr=2682272521
89	17.846534	104.18.94.41	192.168.1.33	TCP	66	443 → 56498 [ACK] Seq=41 Ack=2 Win=17 Len=0 TSval=2682272864 TSecr=29147898
90	18.344979	192.168.1.33	213.180.204.179	TCP	55	52263 → 443 [ACK] Seq=1 Ack=1 Win=511 Len=1
91	18.367793	213.180.204.179	192.168.1.33	TCP	78	443 → 52263 [ACK] Seq=1 Ack=2 Win=166 Len=0 TSval=943384043 TSecr=29103481 SLE=1 SRE=2
92	18.563446	192.168.1.33	88.212.252.206	TCP	55	59596 → 443 [ACK] Seq=1 Ack=1 Win=512 Len=1
93	18.579100	88.212.252.206	192.168.1.33	TCP	78	443 → 59596 [ACK] Seq=1 Ack=2 Win=504 Len=0 TSval=2382853732 TSecr=28923563 SLE=1 SRE=2

Исходные данные пакетов следующие: Захватили сетевой пакет № 36 – ответ от dns сервера Goolge.

Тип пакета: ICMP Echo Reply (эхо ответ на наш запрос в консоли)

Адрес места назначения: От сервера dns Google к локальному компьютеру.
IP компьютера.

Статус: нормальный ответ, подтверждающий работоспособность сетевого соединения.

Канальный уровень (Ethernet II):

- ✓ MAC-адрес получателя: 90:2b:34:58:eb:6b (Сетевая карта)

- ✓ Тип интернет протокола: IPv4

Сетевой уровень (IPv4):

- ✓ IP-адрес отправителя: 8.8.8.8 (dns сервер)
- ✓ IP-адрес получателя: 192.168.1.33 (локальный компьютер_
- ✓ Протокол: ICMP

Транспортный уровень (ICMP):

- ✓ Тип сообщения: 0 (Echo ping reply 0)
- ✓ Код: 0
- ✓ Идентификатор: 1 (0x0001 в hex)
- ✓ Контрольная сумма: 0x5559
- ✓ Номер последовательности: 2 (0x0002 в hex)
- **Анализ полезной нагрузки**

Полезная нагрузка пакета содержит 32 байта данных:

- ✓ Шестнадцатеричное представление: 61 62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76 77 61 62 63 64 65 66 67 68 69
- ✓ ASCII-представление: abcdefghijklmn opqrstuvwxyz abcdefg hi

Данные представляют собой последовательность символов алфавита, которая была отправлена в исходном Echo Request пакете и возвращена без изменений в ответе.

Захватили сетевой пакет № 83 – ответ от сайта Goolge.com.

Общая характеристика пакета:

Тип пакета: ICMP Echo Reply (эхо ответ на наш запрос в консоли)

Размер пакета: 74 байт (592 бита)

Адрес места назначения: От сервера сайта Google к локальному компьютеру. IP компьютера.

Время жизни: 117 мс

Статус: нормальный ответ, подтверждающий работоспособность сетевого соединения.

- **Декомпозиция пакета по уровням OSI**

Канальный уровень (Ethernet II):

- ✓ MAC-адрес отправителя: 28:28:5d:e0:31:70 (роутер Zyxel)
- ✓ MAC-адрес получателя: 90:2b:34:58:eb:6b (Сетевая карта)
- ✓ Тип интернет протокола: IPv4

Сетевой уровень (IPv4):

- ✓ IP-адрес отправителя: 142.251.32.110 (сервер сайта)
- ✓ IP-адрес получателя: 192.168.1.33 (локальный компьютер)
- ✓ Протокол: ICMP

Транспортный уровень (ICMP):

- ✓ Тип сообщения: 0 (Echo ping reply 0)
- ✓ Код: 0
- ✓ Идентификатор: 1 (0x0001 в hex)
- ✓ Контрольная сумма: 0x5556
- ✓ Номер последовательности: 5 (0x0005 в hex)

- **Анализ полезной нагрузки**

Полезная нагрузка пакета содержит 32 байта данных:

- ✓ Шестнадцатеричное представление: 61 62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76 77 61 62 63 64 65 66 67 68 69

- ✓ ASCII-представление: abcdefghijklmn opqrstuvwxyzabcdefghijklmnop hi

Данные представляют собой последовательность символов алфавита, которая была отправлена в исходном Echo Request пакете и возвращена без изменений в ответе.

Выводы

Запросы и ответы пакетов подтверждают успешную двухстороннюю связь между локальным компьютером и сервером Google и dns сервером. Связь с сервисами установлена. Данные *целостны*, возврат исходных данных без изменений свидетельствует об отсутствии ошибок передачи. Все поля пакета соответствуют стандартам Ethernet, IPv4 и ICMP.

Анализ пакета отображает нам корректную работу стека протоколов TCP/IP и успешное выполнение команды ping. Удалённый узел google.com доступен для взаимодействия и работы и сетевое соединение исправно.

POP3.pcap

ФайлПравкаВидЗапускЗахватАнализСтатистикаТелефонияБеспроводная связьИнструментыСправка

Примените фильтр отображения ... <Ctrl-/>

No.	Time	Source	Destination	Protocol	Length	Info
303	4.507933	10.10.162.177	224.0.0.13	IPv2	60	Hello
1949	36.888486	10.10.162.177	224.0.0.13	IPv2	60	Hello
1763	19.171376	94.100.184.77	89.179.4.209	POP	99	S: +OK
1815	20.555349	89.179.4.209	94.100.184.77	POP	95	C: u
1821	20.829148	89.179.4.209	94.100.184.77	POP	95	C: s
1824	21.125933	89.179.4.209	94.100.184.77	POP	95	C: e
1826	21.222285	89.179.4.209	94.100.184.77	POP	95	C: r
1828	21.476172	89.179.4.209	94.100.184.77	POP	95	C: i
1840	22.476376	89.179.4.209	94.100.184.77	POP	95	C: t
1842	22.716393	89.179.4.209	94.100.184.77	POP	95	C: e
1848	23.122358	89.179.4.209	94.100.184.77	POP	95	C: s
1852	23.264932	89.179.4.209	94.100.184.77	POP	95	C: t
1870	26.640940	89.179.4.209	94.100.184.77	POP	95	C: 0
1874	27.159287	89.179.4.209	94.100.184.77	POP	95	C: 0
1881	28.457192	89.179.4.209	94.100.184.77	POP	95	C: 7
1884	28.863845	89.179.4.209	94.100.184.77	POP	95	C: 3
1888	29.261155	89.179.4.209	94.100.184.77	POP	96	C: i
1890	29.301394	94.100.184.77	89.179.4.209	POP	99	S: +OK
1901	31.448764	89.179.4.209	94.100.184.77	POP	95	C: p
1904	31.700636	89.179.4.209	94.100.184.77	POP	95	C: a
1909	32.032750	89.179.4.209	94.100.184.77	POP	95	C: s
1911	32.218612	89.179.4.209	94.100.184.77	POP	95	C: s
1916	33.162925	89.179.4.209	94.100.184.77	POP	95	C: i
1923	35.044780	89.179.4.209	94.100.184.77	POP	95	C: t
1927	35.395671	89.179.4.209	94.100.184.77	POP	95	C: e
1931	35.757621	89.179.4.209	94.100.184.77	POP	95	C: s
1933	35.922206	89.179.4.209	94.100.184.77	POP	95	C: t
1935	36.249642	89.179.4.209	94.100.184.77	POP	95	C: 2
1938	36.457551	89.179.4.209	94.100.184.77	POP	95	C: 2
1941	36.688119	89.179.4.209	94.100.184.77	POP	95	C: 2
1948	36.884183	89.179.4.209	94.100.184.77	POP	95	C: 2
1952	37.092174	89.179.4.209	94.100.184.77	POP	95	C: 2
1954	37.311060	89.179.4.209	94.100.184.77	POP	96	C: i
1956	37.393790	94.100.184.77	89.179.4.209	POP	108	S: +OK Welcome!
1974	40.741947	89.179.4.209	94.100.184.77	POP	95	C: c
1978	41.103878	89.179.4.209	94.100.184.77	POP	95	C: a
1980	41.599376	89.179.4.209	94.100.184.77	POP	95	C: p
1986	42.059569	89.179.4.209	94.100.184.77	POP	95	C: a

Frame 1956: Packet, 108 bytes on wire (864 bits), 108 bytes captured (864 bits)

Ethernet II, Src: DLink_f0:2e:8b (00:19:5b:f0:2e:8b), Dst: Intel_0c:1b:5d (00:11:11:0c:1b:5d)

Internet Protocol Version 4, Src: 89.179.76.133, Dst: 10.117.30.154

User Datagram Protocol, Src Port: 1701, Dst Port: 1701

Layer 2 Tunneling Protocol

Point-to-Point Protocol

Internet Protocol Version 4, Src: 94.100.184.77, Dst: 89.179.4.209

Transmission Control Protocol, Src Port: 110, Dst Port: 1187, Seq: 11, Ack: 32, Len: 14

Post Office Protocol

000000 11 11 0c 1b 5d 00 19 5b f0 2e 8b 00 00 45 00[.....E..

001000 5e 6e d3 00 00 f4 11 08 74 59 b3 4c 85 0a 75 ~^.....~Y~L~u

00201e 9a 06 a5 06 a5 00 4a 00 00 02 00 01 00 01].....

003000 00 ff 03 00 21 45 00 00 36 84 20 40 00 3a 06!E~6~@~:

004047 6c 5e 64 b8 4d 59 b3 04 d1 00 6e 04 a3 f7 48 G!~d~NY~...n~H

0050f9 f4 6a 75 12 64 50 18 11 58 9a 37 00 00 2b 4f ~ju~dP~X~7~+0

00604b 20 57 65 6c 63 6f 6d 65 21 0d 0a K Welcom e!...

Рисунок 11. Анализ файла POP.pcap.

POP3.rcsar					
Файл Правка Вид Запуск Захват Анализ Статистика Телефония Беспроводная связь Инструменты Справка					
pop3					
No.	Time	Source	Destination	Protocol	Length Info
1763	19.171376	94.100.184.77	89.179.4.209	POP	99 S: +OK
1815	20.555349	89.179.4.209	94.100.184.77	POP	95 C: u
1821	20.829148	89.179.4.209	94.100.184.77	POP	95 C: s
1824	21.125933	89.179.4.209	94.100.184.77	POP	95 C: e
1826	21.222285	89.179.4.209	94.100.184.77	POP	95 C: r
1828	21.476172	89.179.4.209	94.100.184.77	POP	95 C: :
1840	22.476376	89.179.4.209	94.100.184.77	POP	95 C: t
1842	22.716393	89.179.4.209	94.100.184.77	POP	95 C: e
1848	23.122350	89.179.4.209	94.100.184.77	POP	95 C: s
1852	23.264932	89.179.4.209	94.100.184.77	POP	95 C: t
1870	26.640940	89.179.4.209	94.100.184.77	POP	95 C: 0
1874	27.159287	89.179.4.209	94.100.184.77	POP	95 C: 0
1881	28.457192	89.179.4.209	94.100.184.77	POP	95 C: 7
1884	28.863845	89.179.4.209	94.100.184.77	POP	95 C: 3
1888	29.261155	89.179.4.209	94.100.184.77	POP	96 C: :
1890	29.301394	94.100.184.77	89.179.4.209	POP	99 S: +OK
1901	31.448764	89.179.4.209	94.100.184.77	POP	95 C: p
1904	31.708636	89.179.4.209	94.100.184.77	POP	95 C: a
1909	32.032750	89.179.4.209	94.100.184.77	POP	95 C: s
1911	32.218612	89.179.4.209	94.100.184.77	POP	95 C: s
1916	33.162925	89.179.4.209	94.100.184.77	POP	95 C: :
1923	35.044780	89.179.4.209	94.100.184.77	POP	95 C: t
1927	35.395671	89.179.4.209	94.100.184.77	POP	95 C: e
1931	35.757621	89.179.4.209	94.100.184.77	POP	95 C: s
1933	35.922286	89.179.4.209	94.100.184.77	POP	95 C: t
1935	36.240642	89.179.4.209	94.100.184.77	POP	95 C: 2
1938	36.457551	89.179.4.209	94.100.184.77	POP	95 C: 2
1941	36.688119	89.179.4.209	94.100.184.77	POP	95 C: 2
1948	36.884183	89.179.4.209	94.100.184.77	POP	95 C: 2
1952	37.092174	89.179.4.209	94.100.184.77	POP	95 C: 2
1954	37.311060	89.179.4.209	94.100.184.77	POP	96 C: :
1956	37.393790	94.100.184.77	89.179.4.209	POP	108 S: +OK Welcome!
1974	40.741347	89.179.4.209	94.100.184.77	POP	95 C: c
1978	41.103878	89.179.4.209	94.100.184.77	POP	95 C: a
1980	41.599376	89.179.4.209	94.100.184.77	POP	95 C: p
1986	42.059569	89.179.4.209	94.100.184.77	POP	95 C: a
1988	42.302657	89.179.4.209	94.100.184.77	POP	96 C: :
1990	42.342852	94.100.184.77	89.179.4.209	POP	123 S: +OK Capability list follows
1994	42.568833	89.179.4.209	94.100.184.77	POP	95 C: s

[Window size scaling factor: -2 (no window scaling used)]
Checksum: 0x35de [unverified]
[Checksum Status: Unverified]
Urgent Pointer: 0
> [Timestamps]
> [SEQ/ACK analysis]
[Client Contiguous Streams: 1]

0000 00 11 11 0c 1b 5d 00 19 5b f0 2e 8b 00 00 45 00 [.....E
0010 00 55 36 1b 00 00 f4 11 c1 35 59 b3 4c 85 0a 75 -U6.....SY.L..u
0020 1e 9a 06 a5 06 a5 00 41 00 00 02 02 00 01 00 01A.....
0030 00 00 ff 03 00 21 45 00 00 2d 84 01 40 00 3a 06IE.....@:..
0040 47 94 5e 64 b0 4d 59 b3 04 d1 00 6e 04 a3 f7 43 6-d.WV.....H
0050 f9 ea 6a 75 12 45 50 18 11 50 35 de 00 00 29 21 -ju-EP..X5...C
0060 0d 0a

Рисунок 12. Анализ файла POP3.rcsar отфильтрованный по pop.

Проанализируем файл POP3 и разберём данные внутри, которые мы обнаружили.

Анализ протокола POP3 и определение учетных данных

1. **Произведём фильтрацию сеанса связи.** Использован был фильтр «pop», который отображает только пакеты, относящиеся к протоколу POP.

2. **Произведём Анализ процесса аутентификации.** В ходе анализа удалось обнаружить следующие этапы аутентификации:

- Команда USER (логин): вводился пользователем в поле для логина.

✓ Временной интервал: 20.555349 - 29.261155 секунд.

✓ Поступательные символы: u s e r t e s t 0 0 7 3

✓ Собранный логин: user test0073

- Команда PASS (пароль): вводился пользователем в поле для пароля
 - ✓ Временной интервал: 31.448764 - 37.311060 секунд
 - ✓ Поступательные символы: p a s s t e s t 2 2 2 2 2
 - ✓ Собранный пароль: pass test22222
- Результаты аутентификации: после ввода логина, пароля, получили сообщение от сервера, что успешно прошли аутентификацию.
 - ✓ После ввода пароля (пакет 1956) сервер отвечает: +OK Welcome!
 - ✓ Это подтверждает успешную аутентификацию с использованием полученных учетных данных.
- Определенные учетные данные: удалось определить следующие данные по учётной записи.
 - ✓ Логин учетной записи: test0073
 - ✓ Пароль учетной записи: test22222
- Дополнительные наблюдения
 - ✓ Протокол POP3 передает данные аутентификации в открытом виде (без шифрования)
 - ✓ Каждый символ логина и пароля передается отдельным пакетом
 - ✓ Сервер 94.100.184.77 принадлежит mail.ru
 - ✓ Клиент имеет IP-адрес 89.179.4.209

Далее проведём отслеживание потока TCP файла POP3, где на протяжении всего сеанса связи, были получены пакеты, содержащие данные из учётного аккаунта, содержащее имя учётной записи, ответы сервера.

USER test0073 – передача логина.

PASS test22222 – передача пароля в открытом виде! Это нарушает все политики безопасности и создаёт очень большую дыру в безопасности. Через

анализатор можно быстро скомпрометировать данные и взломать аккаунт с учётной записью.

+OK *Welcome!* – сообщение об успешной аутентификации

Логин: test0073

Пароль: test22222

CAPA – запрос к серверу о его возможностях.

LIST – список пакетов (2 письма общим размером 83.9 КБ)

STAT – статистика (2 письма, 83968 байт)

QUIT – завершение сеанса

Пакеты, содержащие имя учетной записи: Имя учетной записи test0073 содержится в пакете с командой USER (передача полного логина) и в отдельных пакетах с символами логин.

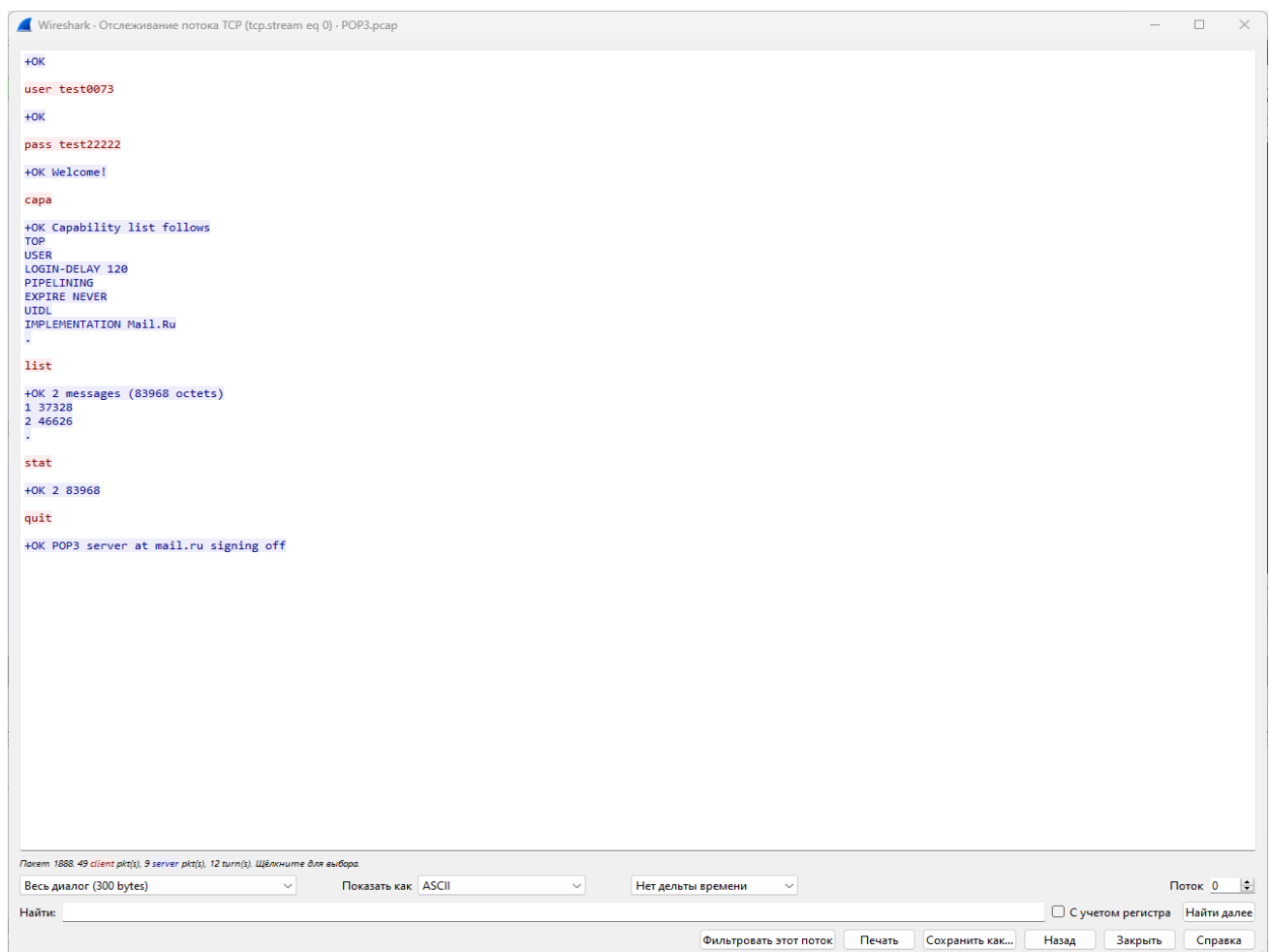


Рисунок 13. Отслеживание связи из общего потока TCP.

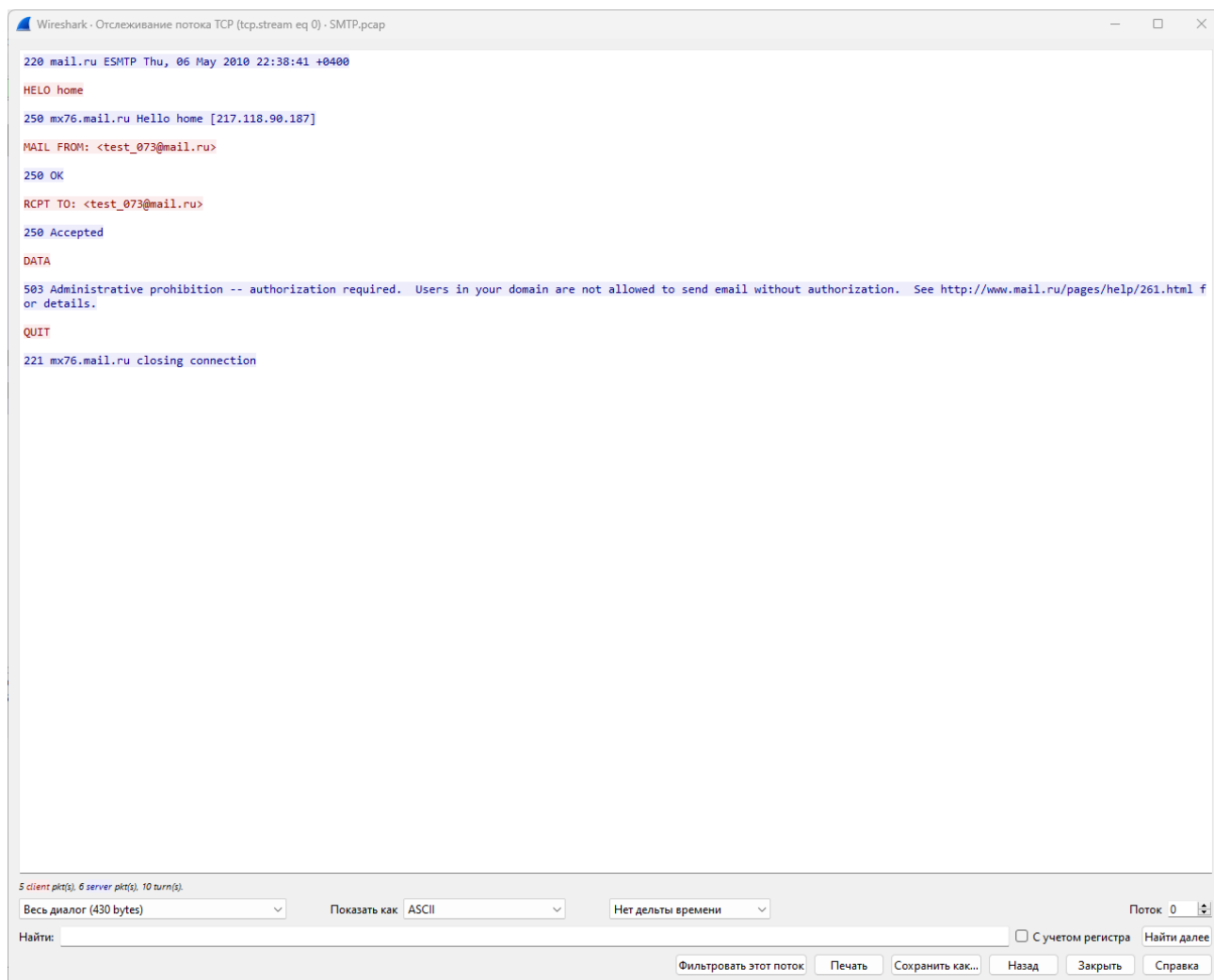


Рисунок 16. Отслеживание связи из общего потока TCP для файла SMTP.pcap.

Анализ SMTP-команд и ответов:

- ✓ 220 mail.ru ESMTP – сервер готов к работе
- ✓ Временная метка: Thu, 06 May 2010 22:38:41 +0400
- ✓ Поддержка расширенного SMTP (ESMTP)
- ✓ HELO home – клиент отображается как "home"
- ✓ Сервер принимает: 250 mx76.mail.ru Hello home [217.118.90.187]
- ✓ IP-адрес клиента: 217.118.90.187

Указание отправителя:

- ✓ Отметка откуда письмо: MAIL FROM <test_073@mail.ru>
- ✓ Сервер подтверждает: сообщение 250 OK

- ✓ Email отправителя: test_073@mail.ru

Указание получателя:

- ✓ Отправитель: RCPT TO: <test_073@mail.ru>
- ✓ Сервер принимает: 250 Accepted
- ✓ Email получателя: test_073@mail.ru (отправка самому себе)

Попытка отправки данных:

- ✓ DATA – команда начала передачи содержимого письма.
- ✓ ОШИБКА: 503 Administrative prohibition -- authorization required.
Нет авторизации.

Причина отказа: Требуется авторизация для отправки писем И политика mail.ru запрещает отправку без аутентификации

Ссылка на документацию: <http://www.mail.ru/pages/help/261.html>

Выводы после анализа:

1. Учетная запись: test_073@mail.ru откуда письмо
2. Ошибка безопасности: пользователь пытается отправить письмо без авторизации
3. Политика mail.ru требует обязательной аутентификации для отправки писем
4. Рекомендация от сервиса использовать SMTP с авторизацией (логин/пароль) или SMTP over SSL

Чем SMTP отличается от POP3:

- **SMTP** – для отправки писем (требуется авторизации от пользователя).
- **POP3** – для получения писем (работает без проблем с теми же учетными данными)

Теперь произведём различную фильтрацию, с включенным захватом кадров. Получим кадры отфильтрованные по ICMP как выяснилось они были получены по тем данным, которые мы предварительно пропинговали и получили ответ Echo.

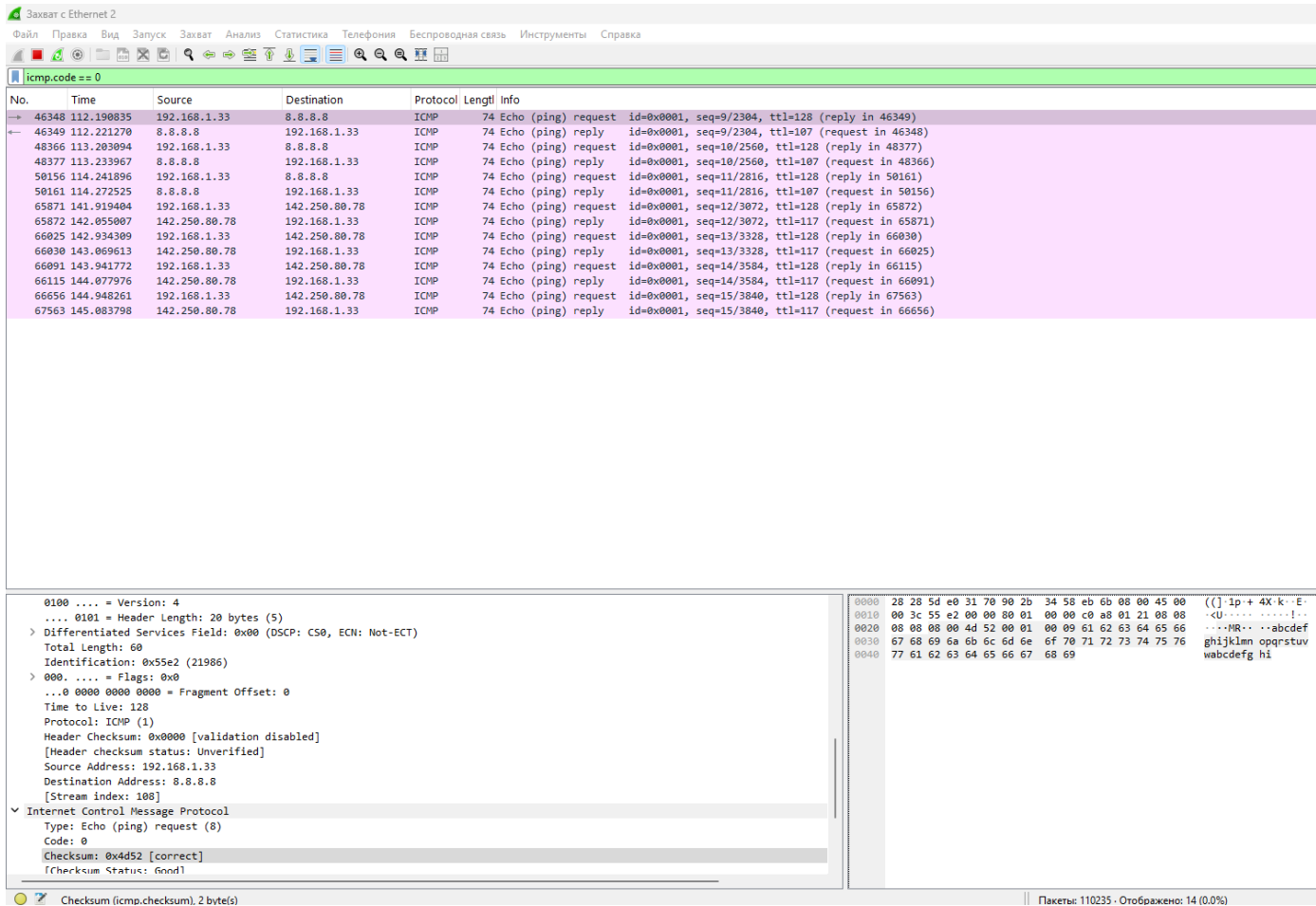


Рисунок 17. ICMP фильтр при захвате кадров на код 0.

Теперь отфильтруем запросы по icmp.code с кодом ответа различным.

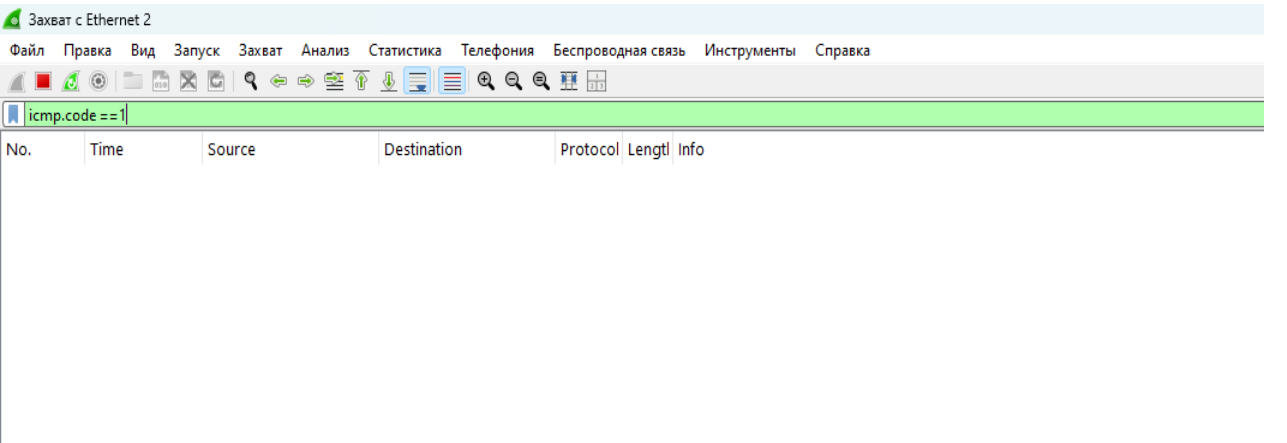


Рисунок 18. ICMP фильтр на запросы по коду 1.

Далее отфильтруем по протоколу ARP предварительно нужно выполнить пинг к адресу 192.168.1.1.

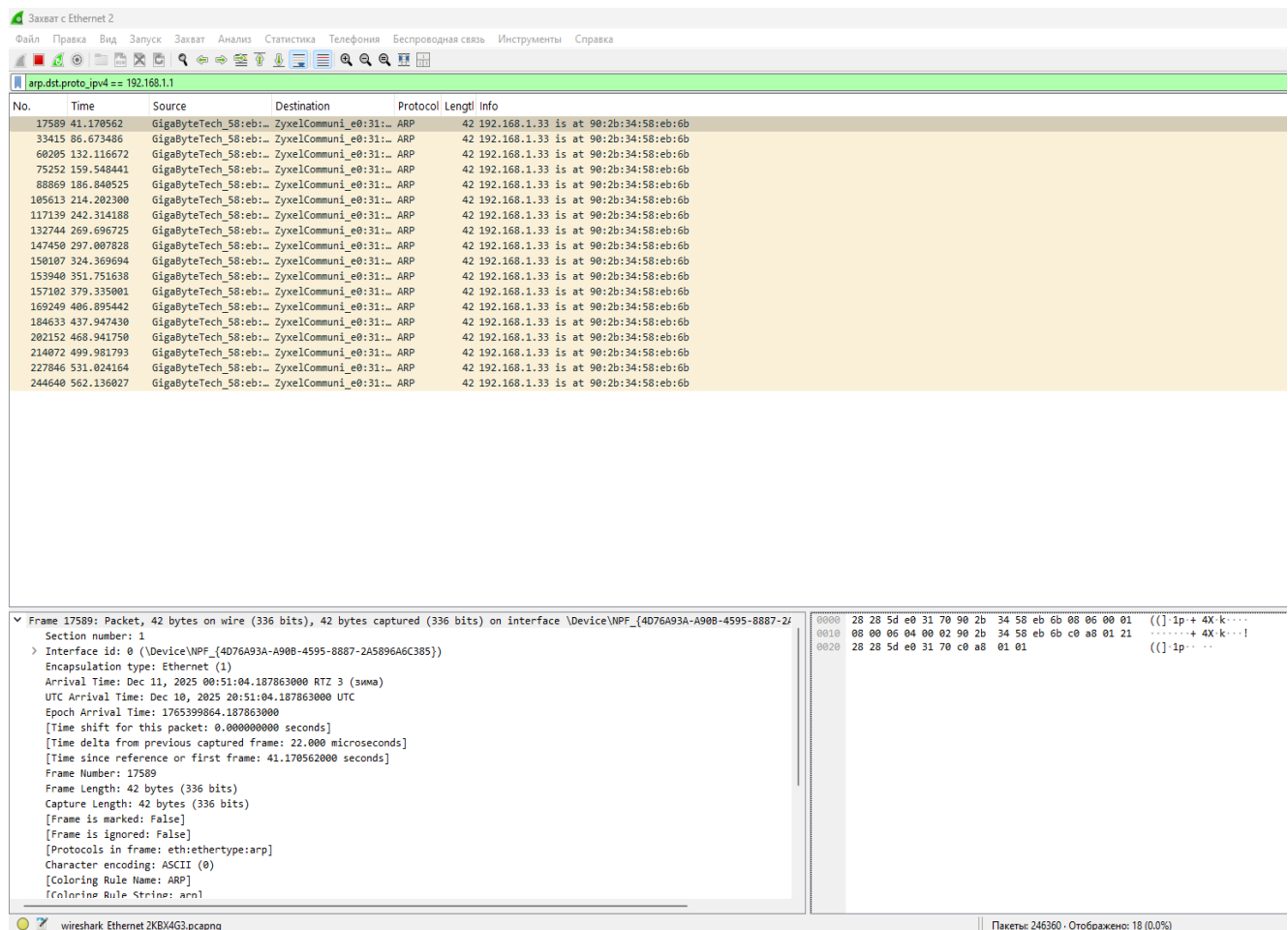


Рисунок 19. Фильтр для выбора протокола ARP после выполнения ping 192.168.1.1.

Теперь произведём фильтрацию или отбор кадров по IP протоколу, протокол ICMP, которые содержатся в поле «Протокол», где адрес назначения является 192.168.1.1. В результате было получено 4 пакета с адресом назначения 192.168.1.1.

А затем произведём фильтрацию по строке «request» в области расшифровки протоколов. Для этого в поле введём фильтрацию кадров/фреймов по слову request.

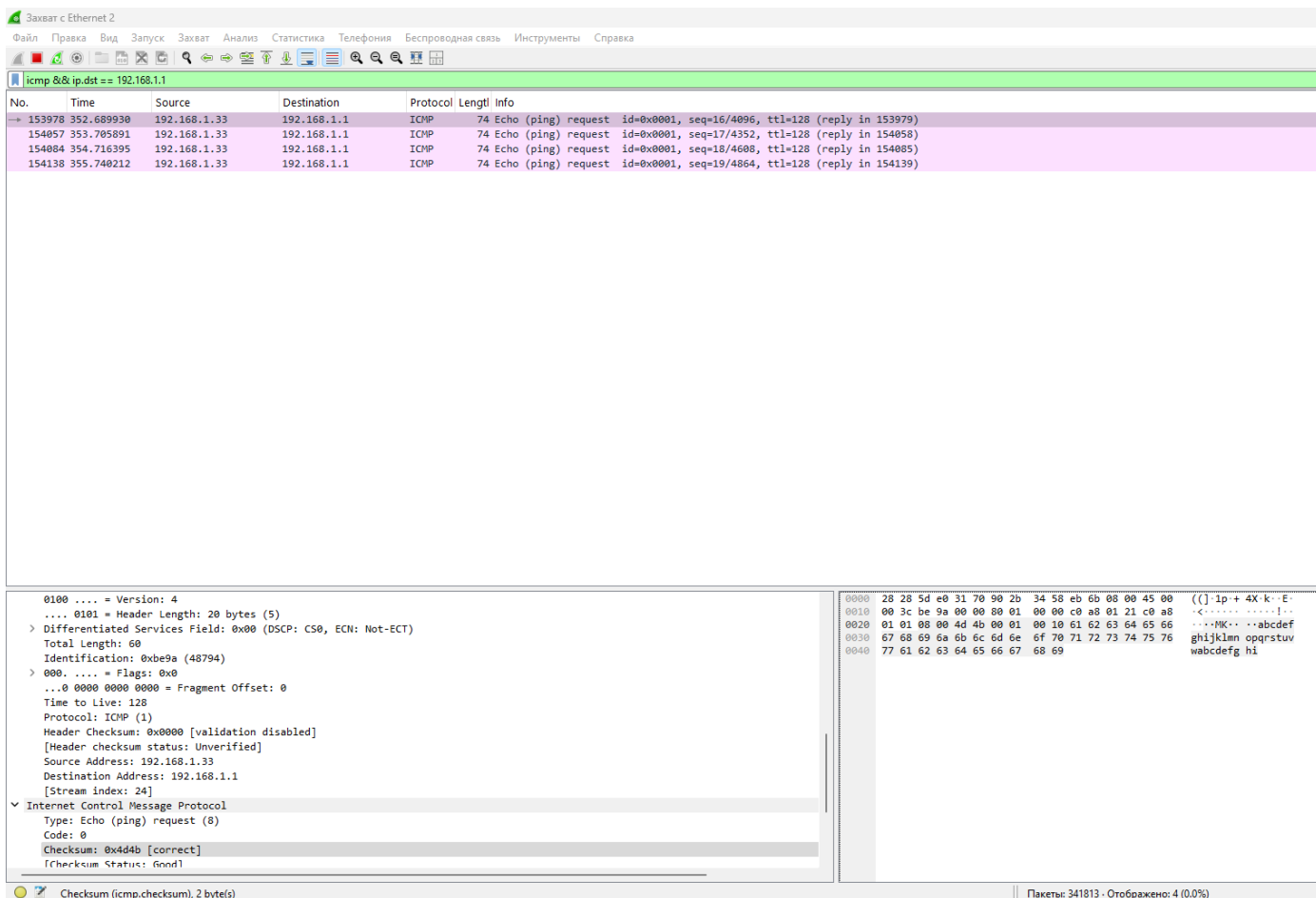


Рисунок 20. Фильтрация кадров, содержащих в поле «Протокол» IP-протокола по протоколу ICMP (адресом назначения 192.168.1.1).

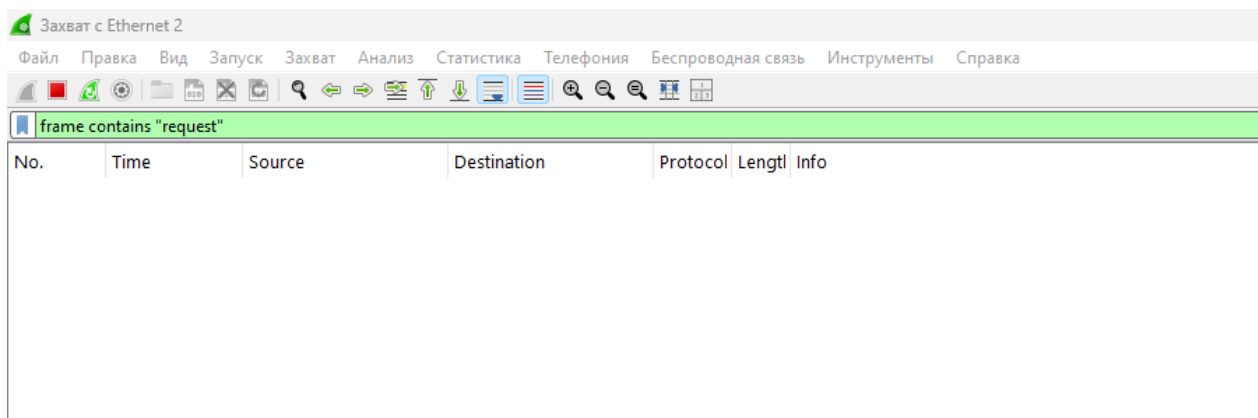


Рисунок 21. Поиск пакетов, по строке «request» в области расшифровки протоколов по кадрам.

Ниже приведу команды, которые производились к сайтам для отображения пакетов по ICMP после того как пропинговали. Пакеты удалось успешно получить по фильтрам.


```
C:\Windows\system32\cmd.e: X + v
Microsoft Windows [Version 10.0.22621.4317]
(c) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

C:\Users\NikBe>ping -n 8.8.8.8
Необходимо указать адрес IP.

C:\Users\NikBe>ping -n 3 8.8.8.8

Обмен пакетами с 8.8.8.8 по с 32 байтами данных:
Ответ от 8.8.8.8: число байт=32 время=30мс TTL=107
Ответ от 8.8.8.8: число байт=32 время=31мс TTL=107
Ответ от 8.8.8.8: число байт=32 время=30мс TTL=107

Статистика Ping для 8.8.8.8:
    Пакетов: отправлено = 3, получено = 3, потеряно = 0
    (0% потерь)
Приблизительное время приема-передачи в мс:
    Минимальное = 30мсек, Максимальное = 31 мсек, Среднее = 30 мсек

C:\Users\NikBe>ping google.com

Обмен пакетами с google.com [142.250.80.78] с 32 байтами данных:
Ответ от 142.250.80.78: число байт=32 время=135мс TTL=117
Ответ от 142.250.80.78: число байт=32 время=135мс TTL=117
Ответ от 142.250.80.78: число байт=32 время=136мс TTL=117
Ответ от 142.250.80.78: число байт=32 время=135мс TTL=117

Статистика Ping для 142.250.80.78:
    Пакетов: отправлено = 4, получено = 4, потеряно = 0
    (0% потерь)
Приблизительное время приема-передачи в мс:
    Минимальное = 135мсек, Максимальное = 136 мсек, Среднее = 135 мсек

C:\Users\NikBe>ping 192.168.1.1

Обмен пакетами с 192.168.1.1 по с 32 байтами данных:
Ответ от 192.168.1.1: число байт=32 время=2мс TTL=64
Ответ от 192.168.1.1: число байт=32 время<1мс TTL=64
Ответ от 192.168.1.1: число байт=32 время=7мс TTL=64
Ответ от 192.168.1.1: число байт=32 время<1мс TTL=64

Статистика Ping для 192.168.1.1:
    Пакетов: отправлено = 4, получено = 4, потеряно = 0
    (0% потерь)
Приблизительное время приема-передачи в мс:
    Минимальное = 0мсек, Максимальное = 7 мсек, Среднее = 2 мсек

C:\Users\NikBe>ftp
```

Рисунок 22. Команды ping в консоли.

Теперь выполним загрузку файла через ftp. Для начала зайдём на ftp.gnu.org, заходим под пользователем анонимным и производим загрузку файла в нужной директории. Для этого в консоли проведём команды к ftp серверу.

```
Администратор: Windows Pc
Windows PowerShell
(C) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

Установите последнюю версию PowerShell для новых функций и улучшения! https://aka.ms/PSWindows

PS C:\Users\NikBe> ftp
ftp> open ftp.gnu.org
Связь с ftp.gnu.org.
220 GNU FTP server ready.
200 Always in UTF8 mode.
Пользователь (ftp.gnu.org:(none)): анонимous
230-NOTICE (Updated October 15 2021):
230-
230-If you maintain scripts used to access ftp.gnu.org over FTP,
230-we strongly encourage you to change them to use HTTPS instead.
230-
230-Eventually we hope to shut down FTP protocol access, but plan
230-to give notice here and other places for several months ahead
230-of time.
230-
230-----
230-
230-Due to U.S. Export Regulations, all cryptographic software on this
230-site is subject to the following legal notice:
230-
230-   This site includes publicly available encryption source code
230-   which, together with object code resulting from the compiling of
230-   publicly available source code, may be exported from the United
230-   States under License Exception "TSU" pursuant to 15 C.F.R. Section
230-   740.13(e).
230-
230-This legal notice applies to cryptographic software only. Please see
230-the Bureau of Industry and Security (www.bxa.doc.gov) for more
230-information about current U.S. regulations.
230 Login successful.
ftp> cd /gnu/hello
250 Directory successfully changed.
ftp> get hello-2.6.tar.gz
500 Illegal PORT command.
425 Use PORT or PASV first.
ftp> quote PASV
227 Entering Passive Mode (209,51,188,20,92,88).
ftp> get hello-2.6.tar.gz
425 Failed to establish connection.
```

Рисунок 23. Командная строка с загрузкой файла с ftp сервера.

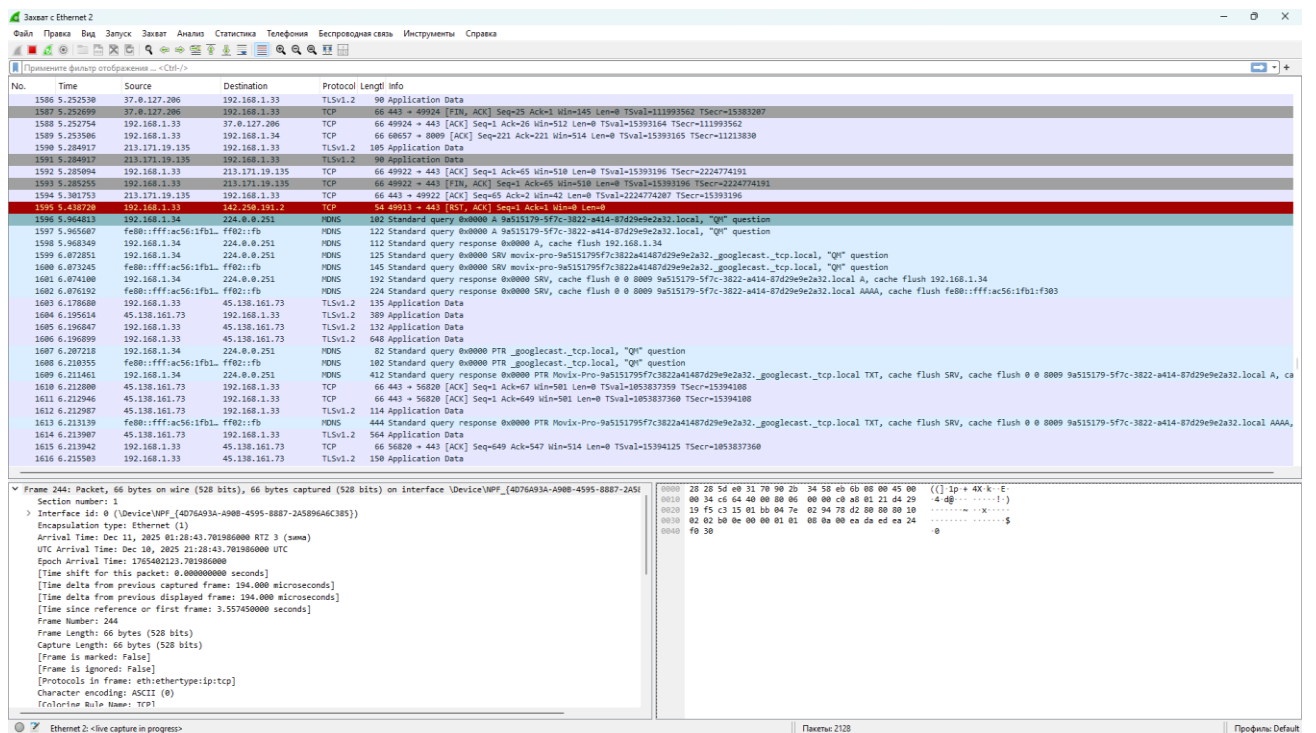


Рисунок 24. Загрузки файла с ftp сервера.

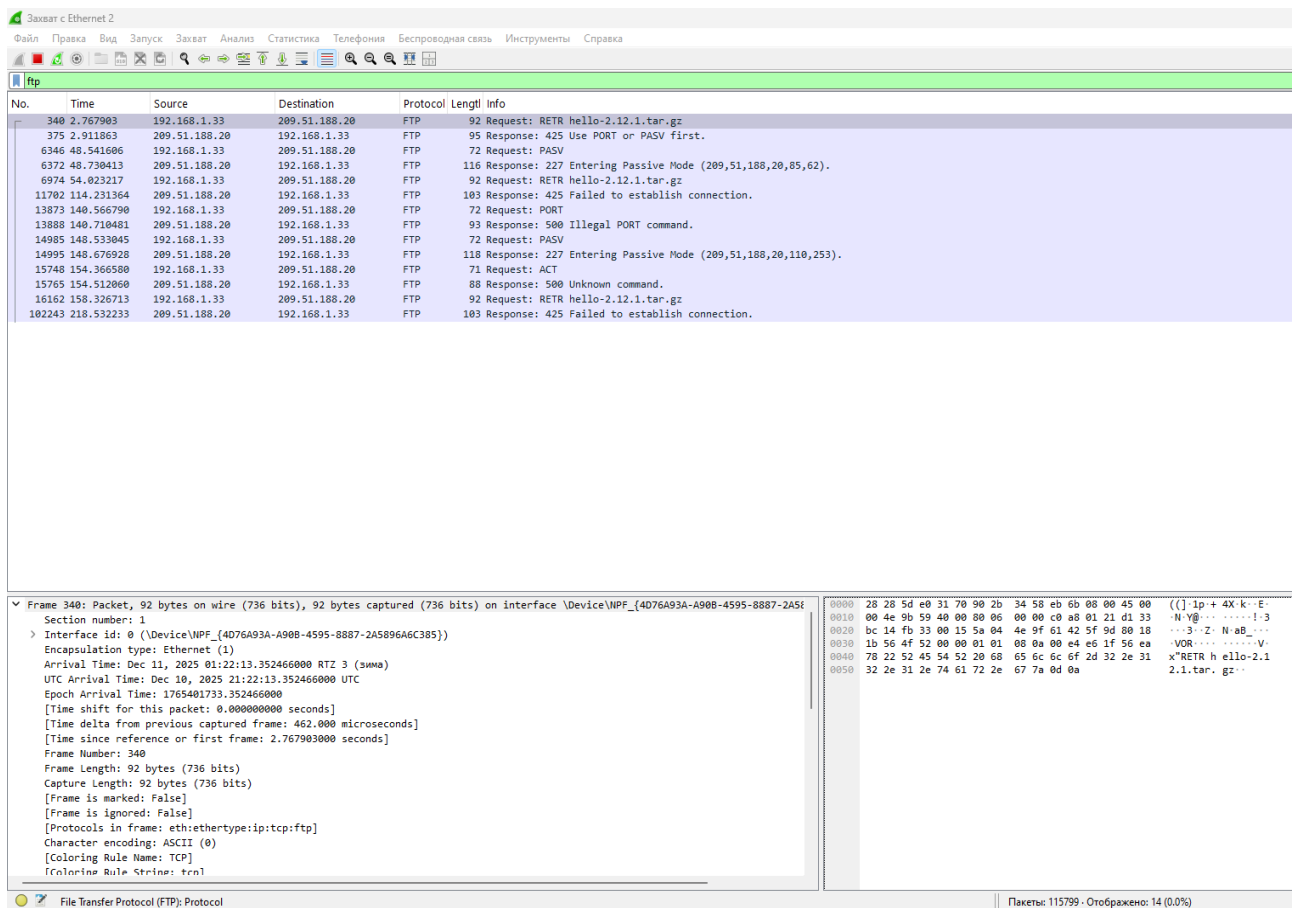


Рисунок 25. ftp запросы к серверу после команд в консоли.

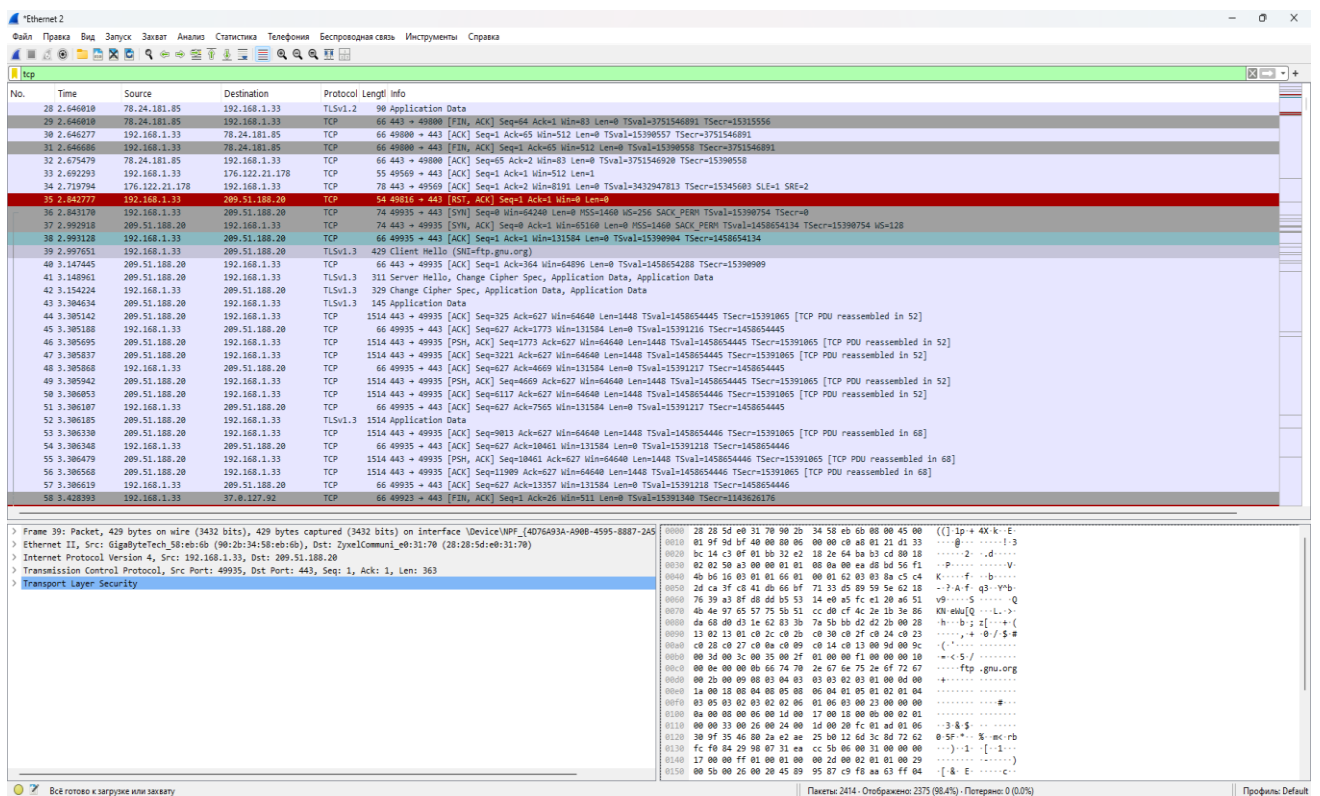


Рисунок 26. Фильтрация по tcp.

Анализ после фильтра tcp

Области изменились после ввода tcp: исчезли "FTP ", "SSL", "TLS" - остались только TCP-заголовки. Видны только порты, Seq/Ack номера, флаги, размер данных (Len).

Фрагментация дейтаграмм: не было фрагментации (все пакеты целые)

TCP-сегменты: всего TCP-сегментов: 98.4% от общего числа пакетов

Сегменты с данными: $Len > 0$ (например, $Len=1448$, $Len=876$)

ACK-сегменты: $Len=0$ (подтверждения)

При применении фильтра tcp в Wireshark отображаются только пакеты на уровне транспортного протокола TCP, что приводит к скрытию информации о прикладных протоколах — таких как FTP, SSL/TLS, HTTP и др. Это означает, что: Структуру TCP-соединений: порты источника/назначения (например, 20-49795 — передача данных FTP, 443-49243 — HTTPS).

TCP-флаги: [ACK], [PSH, ACK], [FIN, ACK] — указывают на установление, передачу данных и завершение соединения.

Номера последовательностей (Seq) и подтверждений (Ack) — позволяют проследить поток данных.

Команды FTP (USER, PASS, RETR, LIST и др.) — они скрыты, так как фильтр tcp не декодирует прикладной уровень. Содержимое файла или его структуру каталога — хотя в некоторых пакетах (особенно с флагом [PSH, ACK]) может быть полезная информация, она отображается только в шестнадцатеричном виде, а не в читаемом формате.

На основании анализа трафика можно сделать следующие выводы: Файл был запрошен, но не получен полностью

В трафике видны TCP-сегменты с данными (Len > 0), что говорит о передаче данных. Однако отсутствие в логах заключительного ответа FTP-сервера (например, 226 Transfer complete) указывает на то, что: Передача могла быть прервана (по таймауту, из-за сетевой ошибки, или пользователем).

Произведём поиск всего количества TCP-соединений для этого отберём все tcp соединения и по портам 20 или 21. Полученные результате приведены на рисунке ниже.

В пункте раздела «Анализ» есть большое количество возможностей и там можно посмотреть различные фильтры. Рассмотрим фильтры отображения: различные параметры фильтрации, макросы для фильтрации, справочная информация, отслеживание потоков TCP и TSL.

Там есть имена фильтров и их выражения удобно для проведения анализа и исследования адресов и пакетов различных.

No.	Time	Source	Destination	Protocol	Length	Info
16248	162.476290	45.138.161.78	192.168.1.33	TLSv1.2	114	Application Data
16249	162.503467	45.138.161.78	192.168.1.33	TLSv1.2	835	Application Data
16250	162.503624	192.168.1.33	45.138.161.78	TCP	66	54823 → 443 [ACK] Seq=42409 Ack=64643 Win=511 Len=0 TSval=18665423 TSecr=3750941009
16251	162.505922	192.168.1.33	45.138.161.78	TLSv1.2	101	Application Data
16252	162.509782	192.168.1.33	45.138.161.78	TCP	66	54828 → 443 [ACK] Seq=4348 Ack=13723 Win=510 Len=0 TSval=18665429 TSecr=3750940991
16253	162.509810	192.168.1.33	87.250.251.66	TCP	55	[TCP Keep-Alive] 63734 → 443 [ACK] Seq=2717 Ack=5637 Win=130048 Len=1
16254	162.512126	192.168.1.33	172.217.105.130	TCP	1466	[TCP Retransmission] 63925 → 443 [ACK] Seq=1 Ack=1 Win=131584 Len=1400 TSval=18665432 TSecr=1752208724
16255	162.520765	192.168.1.33	87.250.251.119	TLSv1.2	578	Application Data
16256	162.520838	192.168.1.33	87.250.251.119	TLSv1.2	105	Application Data
16257	162.521412	192.168.1.33	95.163.52.67	TCP	74	63940 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM TSval=18665440 TSecr=0
16258	162.525905	87.250.251.66	192.168.1.33	TCP	78	[TCP Keep-Alive ACK] 443 → 63734 [ACK] Seq=5637 Ack=2718 Win=42496 Len=0 TSval=1588332693 TSecr=18575324 SLE=2717 SRE=2718
16259	162.538782	95.163.52.67	192.168.1.33	TCP	74	443 → 63940 [SYN, ACK] Seq=0 Ack=1 Win=7240 Len=0 MSS=1460 SACK_PERM TSval=2078573566 TSecr=18665440 WS=16384
16260	162.538944	192.168.1.33	95.163.52.67	TCP	66	63940 → 443 [ACK] Seq=1 Ack=1 Win=131584 Len=0 TSval=18665450 TSecr=2078573566
16261	162.540580	192.168.1.33	95.163.52.67	TLSv1.2	2084	Client Hello (SNI=top-fwz1.mail.ru)
16262	162.543515	87.250.251.119	192.168.1.33	TCP	66	443 → 54815 [ACK] Seq=127719 Ack=51781 Win=164 Len=0 TSval=3199518334 TSecr=18665440
16263	162.543515	192.168.1.33	192.168.1.33	TCP	66	443 → 54815 [ACK] Seq=127719 Ack=51820 Win=164 Len=0 TSval=3199518334 TSecr=18665440
16264	162.544183	87.250.251.119	192.168.1.33	TLSv1.2	122	Application Data
16265	162.544550	192.168.1.33	87.250.251.119	TLSv1.2	105	Application Data
16266	162.548136	87.250.251.119	192.168.1.33	TLSv1.2	451	Application Data
16267	162.549491	192.168.1.33	87.250.251.119	TLSv1.2	101	Application Data
16268	162.550237	192.168.1.33	87.250.250.119	TLSv1.2	97	Application Data
16269	162.558231	95.163.52.67	192.168.1.33	TCP	66	443 → 63940 [ACK] Seq=1 Ack=1449 Win=16384 Len=0 TSval=2078573585 TSecr=18665460
16270	162.558231	95.163.52.67	192.168.1.33	TCP	66	443 → 63940 [ACK] Seq=1 Ack=1999 Win=16384 Len=0 TSval=2078573585 TSecr=18665460
16271	162.558231	95.163.52.67	192.168.1.33	TLSv1.2	216	Server Hello, Change Cipher Spec, Encrypted Handshake Message
16272	162.558231	192.168.1.33	95.163.52.67	TLSv1.2	117	Change Cipher Spec, Encrypted Handshake Message
16273	162.559943	192.168.1.33	95.163.52.67	TLSv1.2	165	Application Data
16274	162.560127	192.168.1.33	95.163.52.67	TLSv1.2	2627	Application Data
16275	162.564454	45.138.161.78	192.168.1.33	TCP	66	443 → 54823 [ACK] Seq=64643 Ack=42444 Win=581 Len=0 TSval=3750941008 TSecr=18665425
16276	162.572966	87.250.251.119	192.168.1.33	TCP	66	443 → 54815 [ACK] Seq=128160 Ack=51894 Win=166 Len=0 TSval=3199518362 TSecr=18665464
16277	162.577119	95.163.52.67	192.168.1.33	TCP	66	443 → 63940 [ACK] Seq=151 Ack=2149 Win=16384 Len=0 TSval=2078573604 TSecr=18665479
16278	162.577119	95.163.52.67	192.168.1.33	TLSv1.2	144	Application Data

> Frame 16273: Packet, 165 bytes on wire (1320 bits), 165 bytes captured (1320 bits) on interface \Device\NPF... [4076A93A-A908-4595-881-...]

> Ethernet II, Src: GigabyteTech 58:eb:6b (90:2b:34:58:eb:6b), Dst: ZyxelCommuni_eb:31:70 (28:28:5d:e0:31:70)

> Internet Protocol Version 4, Src: 192.168.1.33, Dst: 95.163.52.67

> Transmission Control Protocol, Src Port: 63940, Dst Port: 443, Seq: 2050, Ack: 151, Len: 99

Source Port: 63940

Destination Port: 443

[Stream index: 405]

[Stream Packet Number: 9]

> [Conversation completeness: Complete, WITH_DATA (31)]

[TCP Segment Len: 99]

Sequence Number: 2050 (relative sequence number)

Sequence Number (raw): 2670123746

[Next Sequence Number: 2149 (relative sequence number)]

Acknowledgment Number: 151 (relative ack number)

Acknowledgment number (raw): 2736506312

1000 = Header Length: 32 bytes (8)

> Flags: 0x018 (PSH, ACK)

Window: 514

[Calculated window size: 131584]

0000 28 28 5d e0 31 70 90 2b 34 58 eb 6b 08 00 45 00 ([...]p+ 4X k:E

0010 00 97 f4 94 40 00 80 06 00 00 c0 a8 01 21 5f a3 ...@... ..[.

0020 34 43 f9 c4 01 bb 9f 26 da e2 a3 1b c5 c8 80 18 4C...&

0030 02 02 56 39 00 00 01 01 00 0a 81 1c 0d 07 7b 6a ...V9... ..[

0040 72 11 17 03 03 00 5e 00 00 00 00 00 01 2a ...A... ..+

0050 d4 d5 38 47 f6 10 0d da 22 6e 30 a9 56 99 a5 22 ...86... "n0 V..."

0060 b3 c3 df 5b 3c bc bb 1e 18 45 77 0b 22 8b 0c 34 ...[... Ew...4

0070 8a c2 da 18 b9 83 c8 35 18 1a c8 84 e7 b0 9b 5f ...S... ..

0080 68 f6 39 2b df 53 0b 36 bf 4c 96 97 0d 17 85 ca h 9+ S 6 .L.....

0090 1c e7 4c aa 4a 5c 2d 4f 65 fb 93 92 e2 8c 4d 99 ...L\)-0 e-9:M

00a0 dd 10 23 ba ca ...#...

Пакеты: 228449 · Отображено: 225009 (98.5%) · Потеряно: 0 (0.0%)

Рисунок 27. Определение количества TCP соединений.

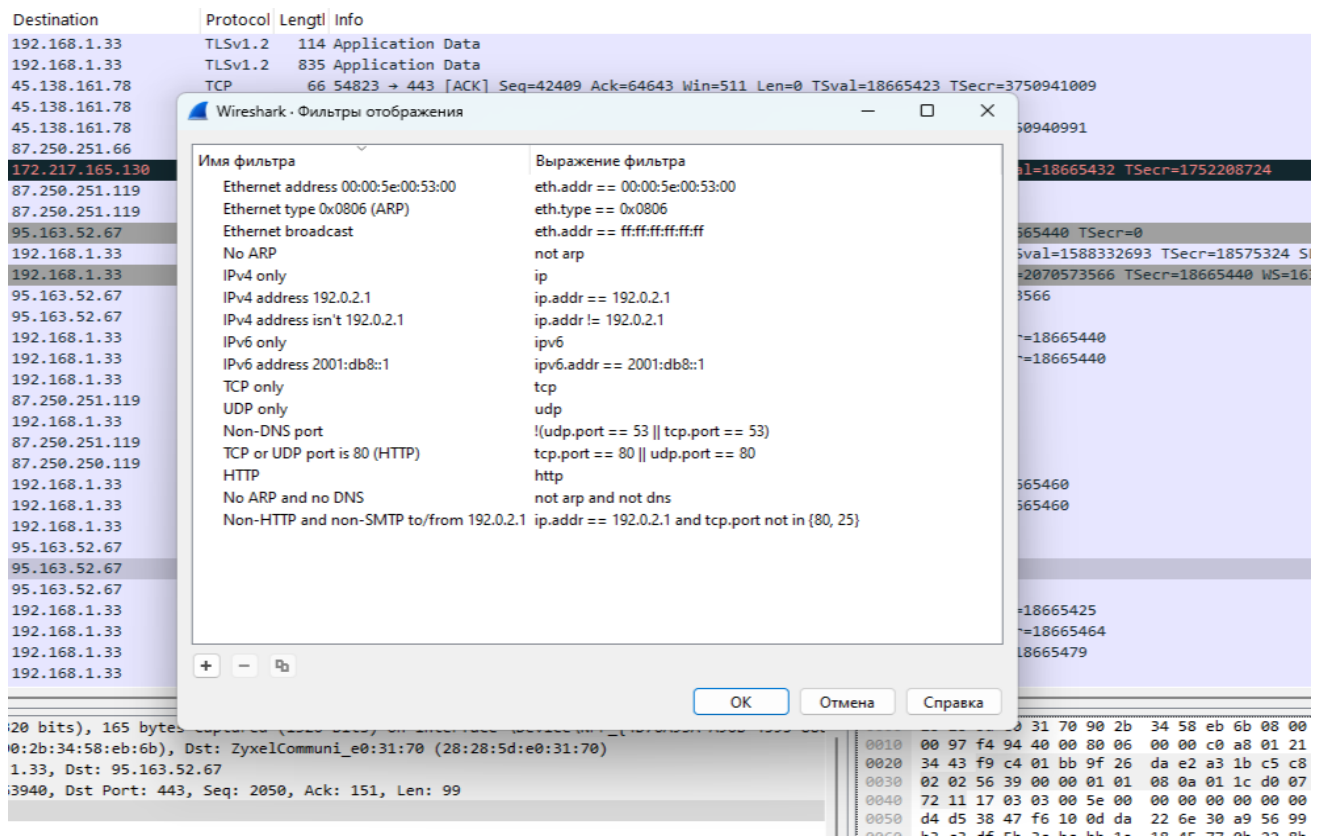


Рисунок 28. Раздел в меню «Фильтры отображения».

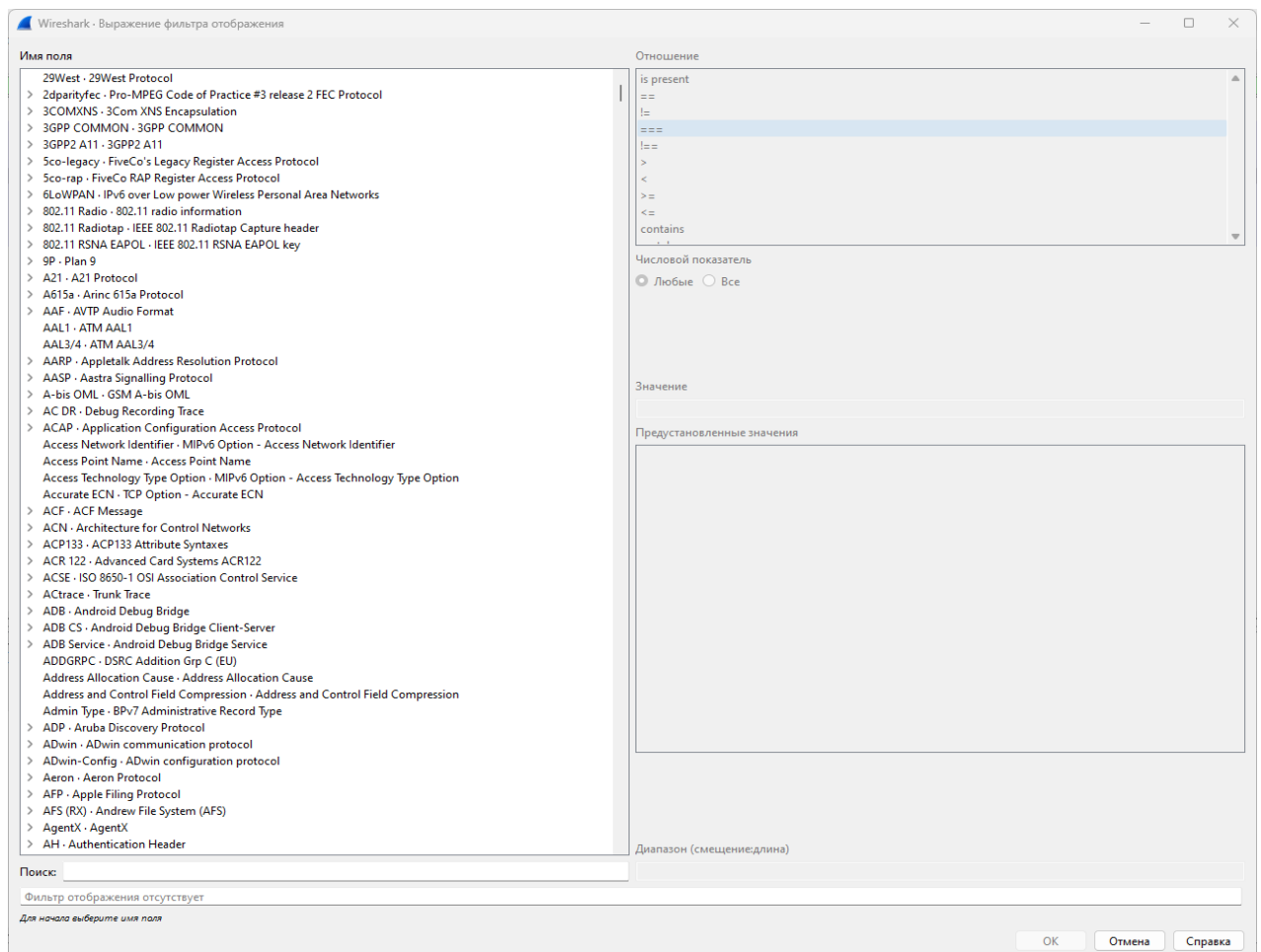


Рисунок 29. Раздел меню «Выражения фильтра отображения».

Раздел фильтрации в Wireshark — это ключевой инструмент для анализа сетевого трафика, позволяющий отсеивать ненужные пакеты и фокусироваться на интересующих данных.

Основная цель фильтрации — это показывать только те пакеты, которые соответствуют заданным критериям, а остальные — скрывать. Это необходимо, потому что при захвате трафика (особенно в реальной сети) может быть тысячи или миллионы пакетов, и без фильтрации найти нужную информацию практически невозможно.

В Wireshark существует два основных типа фильтров:

1. Фильтр отображения (Display Filter) — самый часто используемый
 - Применяется после захвата.
 - Не удаляет пакеты из файла — просто скрывает их.
 - Можно изменять/обновлять в любое время.
 - Вводится в строку "Фильтр отображения" вверху окна Wireshark.
2. Фильтр захвата (Capture Filter) — применяется при записи
 - Применяется до начала захвата.
 - Пакеты, не прошедшие фильтр, не записываются в файл вообще.
 - Используется язык BPF (Berkeley Packet Filter) — более ограниченный, чем Display Filter.

У Wireshark большие возможности для фильтрации, которые позволяет комбинировать поля, операторы, функции, применять логические операторы, группировку, операторы сравнения и задавать в полях эти условия.

Как работать с фильтрами на практике (по шагам):

- Шаг 1: Захват трафика

Начните захват, подождите, пока накопится трафик.

- Шаг 2: Откройте фильтр отображения

Введите фильтр в строку вверху: например, `http`. Нажмите Enter — список пакетов обновится.

- Шаг 3: Уточнение фильтра

Щёлкните правой кнопкой по полю в пакете Apply as Filter выберите Selected или Not Selected.

- Шаг 4: Сохранение фильтров

В окне «Фильтры отображения» можно сохранить часто используемые фильтры. Нажмите + (введите имя, добавьте выражение).

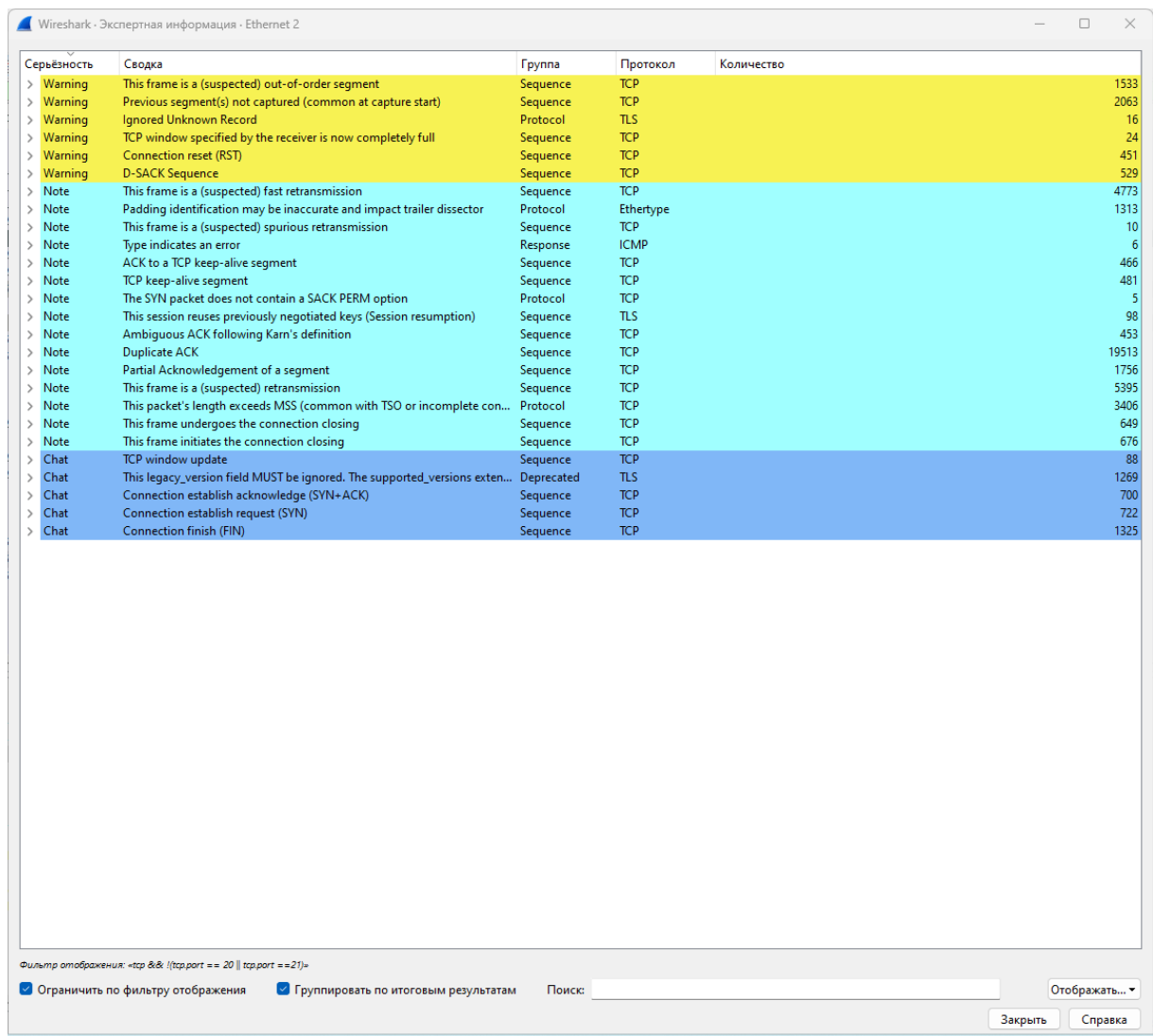


Рисунок 30. Раздел «Экспертная информация».

В разделе экспертной информации можно получать подробную информацию по трафику.

Экспертная информация (Expert Info) в Wireshark — это встроенный анализатор сетевого трафика, который автоматически выявляет аномалии, потенциальные ошибки, нестандартное поведение и важные события в

захваченных пакетах, группируя их по уровню критичности: Warning (предупреждение), Note (замечание), Chat (диалог/норма) и Error (ошибка). Эта функция нужна для того, чтобы аналитик мог быстро обнаружить проблемы в сети (потерю пакетов, перезапуск соединений, неправильную последовательность TCP-сегментов, устаревшие протоколы или подозрительные действия). Без необходимости вручную просматривать каждый пакет. Эта функция экономит время, помогает диагностировать сетевые сбои, оптимизировать производительность, выявлять признаки атак или неправильную конфигурацию устройств, особенно когда трафик большой и сложный. Вместо того чтобы листать тысячи строк, можно сразу увидеть, где что пошло не так и на что стоит обратить внимание в первую очередь.

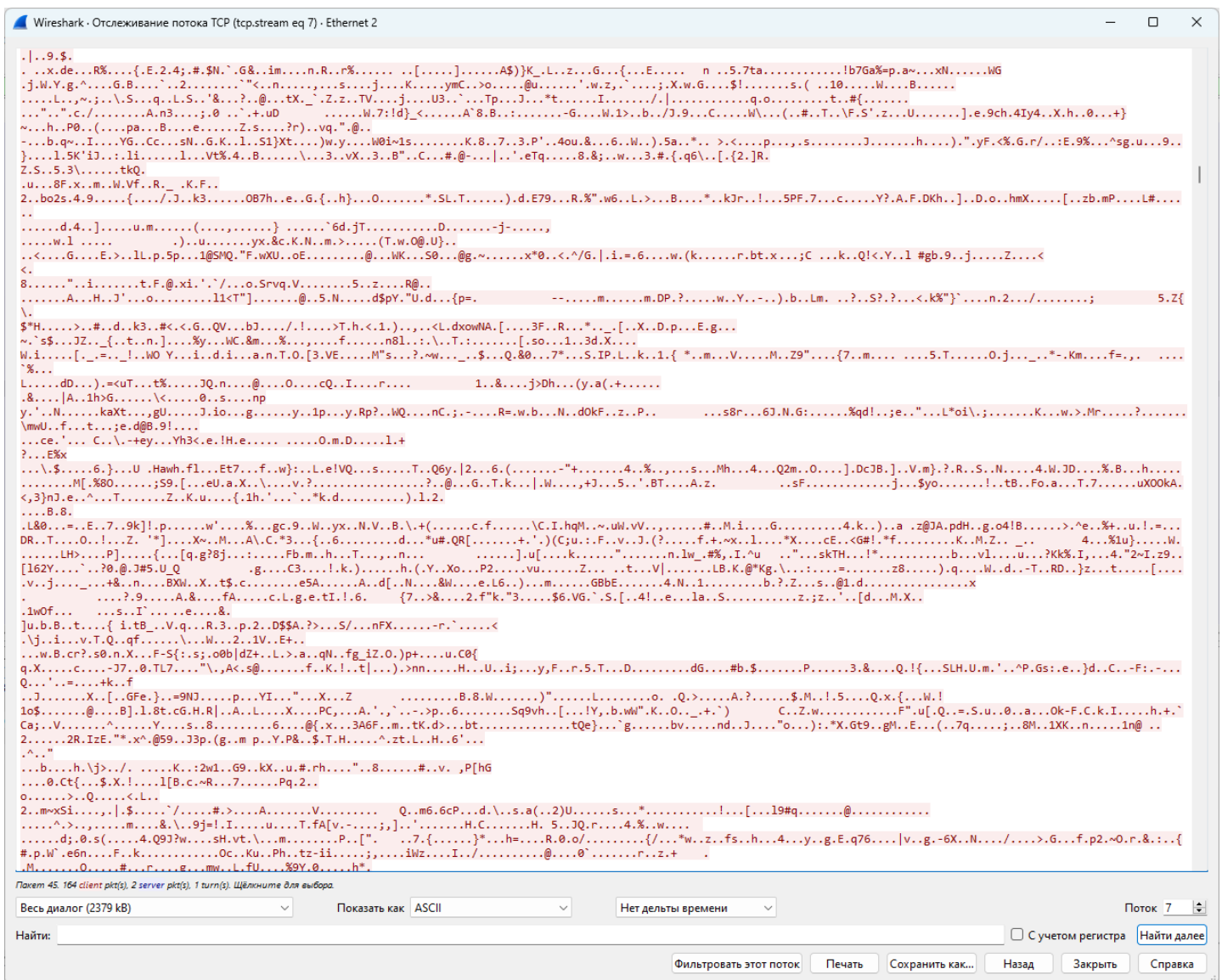


Рисунок 31. Процесс «Отслеживание потока TCP» из меню выборка.

Отслеживание потока – это функция, которая собирает все пакеты одного сеанса (TCP, UDP, TLS, HTTP) в единый поток, как будто вы читаете лог общения между двумя сторонами — как в терминале или чате. Все данные, переданные в обоих направлениях (клиент – сервер и сервер – клиент). Отслеживание позволяет:

- Просмотреть содержимое HTTP-запросов и ответов (заголовки, тело, параметры).
- Анализировать FTP, SMTP, POP3, IMAP (команды и ответы сервера).
- Изучить TLS-трафик (есть ключи, Wireshark расшифрует и покажет plaintext).
- Выявить передачу чувствительной информации (логины, пароли, токены, если не шифруются).
- Дебаггинг приложений — посмотреть, что именно отправляет клиент и получает сервер.
- Обнаружение атак — например, SQL-инъекции, XSS, команды shell в HTTP-запросах.

Как работает? Правок кнопкой мыши по любому пакету TCP-соединения, выбираем отслеживание TCP пакета и в диалоговом окне получаем всю информацию.

Статистика сеансов (Statistics – Conversations) Это сводная таблица всех сетевых "разговоров" (сеансов) между парами устройств за весь период захвата — по протоколам: Ethernet, IPv4, IPv6, TCP, UDP, ICMP.

- Выявить "тяжелые" сеансы (какие соединения генерируют больше всего трафика или пакетов_.
- Найти "подозрительные" пары (пример, один IP активно общается с множеством других и возможна атака).

- Оценить производительность — по скорости передачи данных (bit/s) можно понять, есть ли узкие места.
- Упростить анализ — вместо просмотра тысяч пакетов вы видите агрегированные данные по каждому сеансу.
- Сравнить направления (если сервер отправляет много данных, а клиент мало, это нормально для HTTP, но подозрительно для FTP или SSH).

Адрес А	Порт А	Адрес В	Порт В	Пакеты	Байты	ИД потока	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Отн. время начала	Продолжительность	Bits/s A → B	Bits/s B → A	Потоки
2.20.142.2	443	192.168.1.33	59337	0	0 байты	3	0	0 байты	0	0 байты	5.809374	0.0001	0 байты/с	0 байты/с	1
2.20.142.2	443	192.168.1.33	59336	0	0 байты	52	0	0 байты	0	0 байты	105.977156	0.0003	0 байты/с	0 байты/с	1
13.107.5.93	443	192.168.1.33	51468	0	0 байты	46	0	0 байты	0	0 байты	55.676107	0.0000	0 байты/с	0 байты/с	0
13.107.139.11	443	192.168.1.33	49753	0	0 байты	9	0	0 байты	0	0 байты	12.347232	0.0000	0 байты/с	0 байты/с	0
13.107.139.11	443	192.168.1.33	49754	0	0 байты	12	0	0 байты	0	0 байты	15.529642	0.0000	0 байты/с	0 байты/с	0
13.107.246.44	443	192.168.1.33	49696	0	0 байты	4	0	0 байты	0	0 байты	5.829121	0.0569	0 байты/с	0 байты/с	1
13.107.246.44	443	192.168.1.33	49733	0	0 байты	19	0	0 байты	0	0 байты	24.213512	0.0557	0 байты/с	0 байты/с	1
20.199.181.0	443	192.168.1.33	49715	0	0 байты	53	0	0 байты	0	0 байты	113.717537	0.0000	0 байты/с	0 байты/с	0
52.109.28.46	443	192.168.1.33	49699	0	0 байты	40	0	0 байты	0	0 байты	48.666689	0.0600	0 байты/с	0 байты/с	1
52.109.68.130	443	192.168.1.33	49706	0	0 байты	30	0	0 байты	0	0 байты	40.647814	0.0000	0 байты/с	0 байты/с	0
52.123.128.14	443	192.168.1.33	49760	0	0 байты	37	0	0 байты	0	0 байты	46.319454	0.0000	0 байты/с	0 байты/с	0
92.122.215.2	443	192.168.1.33	49702	0	0 байты	25	0	0 байты	0	0 байты	36.415258	0.0003	0 байты/с	0 байты/с	1
150.171.22.11	443	192.168.1.33	49731	0	0 байты	1	0	0 байты	0	0 байты	0.442378	0.0000	0 байты/с	0 байты/с	0
192.168.1.33	49701	2.16.69.71	443	0	0 байты	22	0	0 байты	0	0 байты	25.945387	0.0349	0 байты/с	0 байты/с	1
192.168.1.33	49726	2.23.89.83	443	0	0 байты	34	0	0 байты	0	0 байты	41.043771	0.0339	0 байты/с	0 байты/с	1
192.168.1.33	49671	3.164.230.8	443	0	0 байты	13	0	0 байты	0	0 байты	16.246369	0.0341	0 байты/с	0 байты/с	0
192.168.1.33	57304	4.207.247.139	443	0	0 байты	0	0	0 байты	0	0 байты	0.270038	2.0025	0 байты/с	0 байты/с	0
192.168.1.33	57306	4.207.247.139	443	0	0 байты	5	0	0 байты	0	0 байты	8.333059	3.0225	0 байты/с	0 байты/с	0
192.168.1.33	57308	4.207.247.139	443	0	0 байты	11	0	0 байты	0	0 байты	15.369143	3.0119	0 байты/с	0 байты/с	0
192.168.1.33	57313	4.207.247.139	443	0	0 байты	26	0	0 байты	0	0 байты	37.419736	3.0046	0 байты/с	0 байты/с	0
192.168.1.33	57314	4.207.247.139	443	0	0 байты	36	0	0 байты	0	0 байты	44.445111	3.0145	0 байты/с	0 байты/с	0
192.168.1.33	57317	4.207.247.139	443	0	0 байты	45	0	0 байты	0	0 байты	53.889009	3.0144	0 байты/с	0 байты/с	0
192.168.1.33	57318	4.207.247.139	443	0	0 байты	47	0	0 байты	0	0 байты	60.935560	3.0213	0 байты/с	0 байты/с	0
192.168.1.33	57321	4.207.247.139	443	0	0 байты	50	0	0 байты	0	0 байты	94.495787	3.0107	0 байты/с	0 байты/с	0
192.168.1.33	57322	4.207.247.139	443	0	0 байты	51	0	0 байты	0	0 байты	101.518095	0.0000	0 байты/с	0 байты/с	0
192.168.1.33	49719	13.107.139.11	443	0	0 байты	7	0	0 байты	0	0 байты	9.999721	60.8917	0 байты/с	0 байты/с	2
192.168.1.33	49739	13.107.139.11	443	0	0 байты	6	0	0 байты	0	0 байты	9.999494	0.0388	0 байты/с	0 байты/с	0
192.168.1.33	57319	13.107.139.11	443	0	0 байты	48	0	0 байты	0	0 байты	70.852326	1.1563	0 байты/с	0 байты/с	8
192.168.1.33	49698	20.54.232.160	443	0	0 байты	20	0	0 байты	0	0 байты	25.299621	0.0525	0 байты/с	0 байты/с	0
192.168.1.33	49732	20.135.6.9	443	0	0 байты	33	0	0 байты	0	0 байты	40.872855	0.1083	0 байты/с	0 байты/с	0
192.168.1.33	57297	20.189.173.3	443	0	0 байты	44	0	0 байты	0	0 байты	51.857675	0.0000	0 байты/с	0 байты/с	0
192.168.1.33	57298	20.189.173.3	443	0	0 байты	41	0	0 байты	0	0 байты	51.855672	0.0000	0 байты/с	0 байты/с	0
192.168.1.33	57299	20.189.173.3	443	0	0 байты	42	0	0 байты	0	0 байты	51.855905	0.0000	0 байты/с	0 байты/с	0
192.168.1.33	49683	20.190.181.0	443	0	0 байты	15	0	0 байты	0	0 байты	18.461210	0.0375	0 байты/с	0 байты/с	0
192.168.1.33	49735	20.199.38.43	443	0	0 байты	32	0	0 байты	0	0 байты	40.672660	0.0621	0 байты/с	0 байты/с	0
192.168.1.33	51467	20.199.38.43	443	0	0 байты	38	0	0 байты	0	0 байты	46.425720	0.0000	0 байты/с	0 байты/с	0

Рисунок 32. Статистика сеансов TCP.

Далее откроем статистику сеансов TCP, где выберем первый сеанс из меню фильтров Apply as Filter -> Selected -> A<->B. Далее получим списки пакетов, которые отфильтруются. Мы применили фильтр отображения, который показывает только те пакеты, которые участвуют в этом конкретном сетевом разговоре между двумя устройствами.

Фильтрация A – B сужает весь захваченный трафик до одного логического соединения, чтобы вы могли: сосредоточиться на одном диалоге, не отвлекаться на фоновый шум (DNS, ARP, другие сеансы) и легко анализировать последовательность команд и ответов между клиентом и сервером.

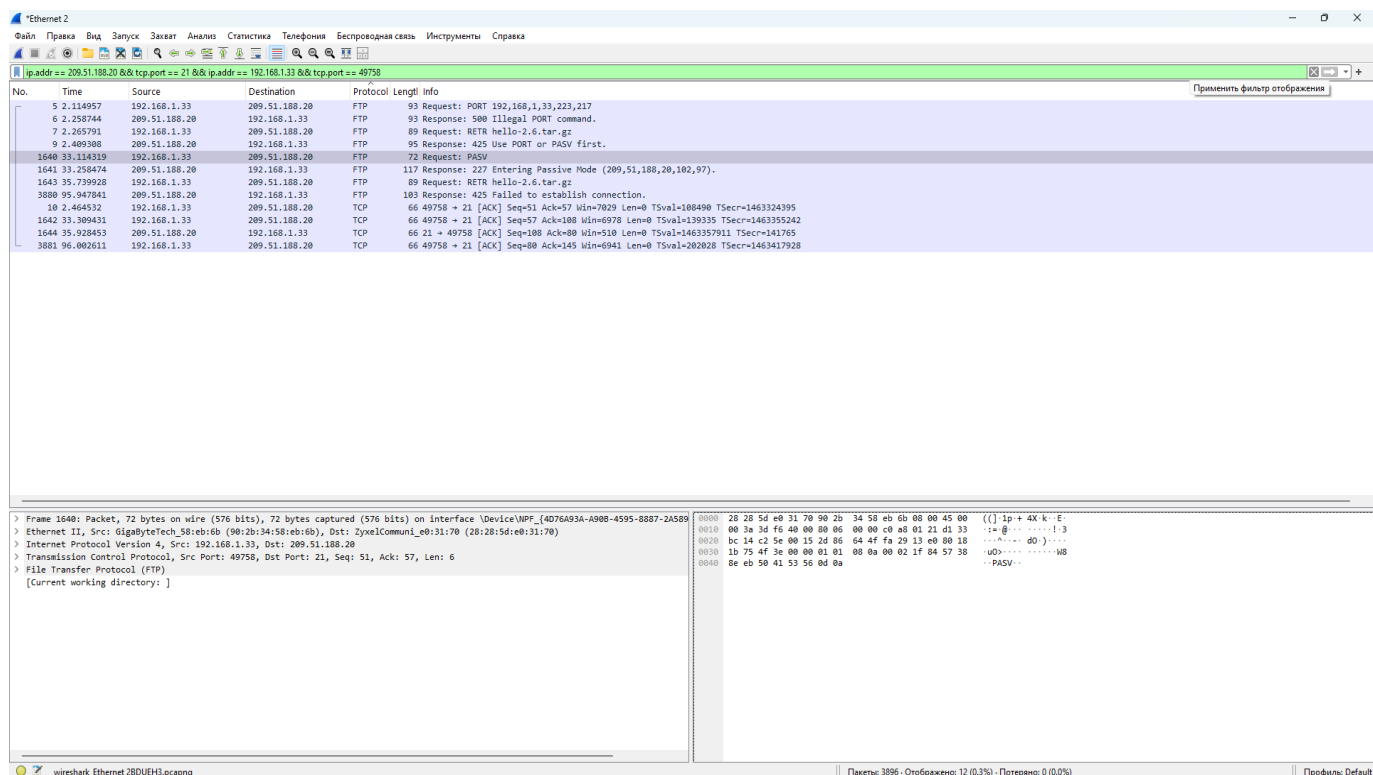


Рисунок 33. Фильтрация для сеанса Apply as Filter Selected – A <-> B.

Выводы

В ходе выполнения лабораторной работы были проанализированы сетевые захваты POP.рсар и SMTP.рсар, в которых наглядно продемонстрирована передача конфиденциальной информации в открытом, нешифрованном виде. В протоколе POP3 при аутентификации пользователя логин и пароль передаются в текстовом формате без какого-либо шифрования, что делает их легко доступными для перехвата злоумышленником с использованием простых инструментов анализа трафика, таких как Wireshark. Это в корне *нарушает политику информационной безопасности* и подвергает угрозе компрометированы данных и кражу личных данных из учётной записи.

Аналогичная уязвимость выявлена в протоколе SMTP — даже при попытке отправки письма сервер явно указывает на необходимость авторизации, однако сам процесс аутентификации (если бы он был реализован) также происходил бы в открытом виде, поскольку используется незащищённый канал связи.

Полученные результаты подтверждают высокую степень риска при использовании устаревших или незащищённых протоколов электронной почты в современных сетях: любое соединение, основанное на POP3 или SMTP без TLS/SSL, является потенциально уязвимым к атакам типа **"перехват трафика" (sniffing)**, что позволяет злоумышленнику получить доступ к учётным данным пользователей, содержимому писем и другим чувствительным данным.

Это противоречит базовым принципам информационной безопасности (ИБ) – *конфиденциальности, целостности и доступности* данных. Для минимизации рисков рекомендуется немедленно переводить все почтовые сервисы на защищённые версии протоколов (IMAP4 over SSL/TLS и SMTP over SSL/TLS (SMTPS)). Также можно использовать современные методы аутентификации с обязательным шифрованием канала.

Изученные на предыдущих лабораторных методы шифрования и хэширования успешно бы помогли защитить данные при передаче по сетям и обеспечить безопасность трафика. **Шифрование** (например, TLS/SSL) обеспечивает конфиденциальность передаваемых данных: даже если трафик перехвачен, содержимое остаётся недоступным без соответствующего ключа. В противоположность этому, хэширование (например, хранение паролей в виде хэшей на сервере) предназначено для проверки целостности и аутентификации, но не защищает данные при передаче по сети.

В файлах POP.rcar и SMTP.rcar учётные данные передавались в открытом виде (ни шифрование, ни даже хэширование при передаче не применялись), что делает возможным мгновенное извлечение логина и пароля из трафика.

Анализ в Wireshark также показывает, какие атаки возможны при несоблюдении принципов защиты данных. Помимо простого перехвата учётных данных (credential sniffing), злоумышленник может выполнить

повторную отправку (replay attack) аутентификационных пакетов, если протокол не использует одноразовые токены или временные метки. При работе с незащищёнными веб-формами или API можно извлечь cookie-файлы, токены сессий, персональные данные или даже файлы, передаваемые по HTTP, FTP или SMTP. Если в трафике присутствует базовая HTTP-аутентификация (Basic Auth), логин и пароль кодируются лишь в Base64 — что не является шифрованием и легко декодируется.

Wireshark позволяет не только обнаружить такие уязвимости, но и воссоздать полный диалог между клиентом и сервером, что даёт полную картину компрометации.

Таким образом, анализ захваченного трафика наглядно показал, что в условиях отсутствия шифрования сетевой трафик является полностью прозрачным для внешнего наблюдателя, что создаёт реальную угрозу кражи корпоративной и персональной информации. Данная лабораторная работа подчеркивает критическую важность применения механизмов защиты на транспортном уровне (TLS) для всех сервисов и необходимости отказа от использования незащищённых протоколов в производственной среде.

Обработки паролей лучше производить через безопасные механизмы аутентификации (OAuth) и хранения паролей на стороне сервера исключительно в виде криптографически стойких хэшей с солью (bcrypt, Argon2).

В отчётах по пентесту¹ или аудиту безопасности в инфраструктуре систем, именно такие находки, как передача данных в открытом виде, классифицируются как высокочитичные риски и требуют немедленного устранения!

¹ **Испытание на проникновение** (пентест, пентестинг) — метод оценки безопасности компьютерных систем или сетей путём моделирования атаки злоумышленника. Процесс включает в себя активный анализ системы на наличие потенциальных уязвимостей, которые могут спровоцировать некорректную работу целевой системы, либо полный отказ в обслуживании.

Заключение

В ходе выполнения лабораторной работы по захвату и анализу сетевого трафика с использованием программы Wireshark были освоены ключевые принципы работы с сетевыми анализаторами и углублены знания о структуре и функционировании протоколов стека TCP/IP, включая ICMP, ARP, TCP, IP, а также прикладные протоколы SMTP и POP3.

Были изучены основные компоненты интерфейса Wireshark: *фильтры захвата и отображения, буфер пакетов, декодер протоколов*, панель шестнадцатеричного представления и модуль статистики. Практически освоены методы настройки параметров захвата, включая выбор сетевого интерфейса, режима захвата, ограничения объёма данных и применения фильтров.

В ходе выполнения лабораторной работы, мне удалось успешно:

- Произвести захват и анализ трафика, сгенерированный командой ping, что позволило изучить структуру ICMP-сообщений и механизм разрешения адресов с помощью ARP
- Удалось успешно проанализировать заранее сохранённые файлы трафика POP3.pcap и SMTP.pcap в результате чего были выявлены учётные данные (включая пароль), команды протоколов и особенности взаимодействия с почтовым сервером.
- Исследовали FTP-трафик, включая передачу файлов и структуры каталогов, а также фрагментацию данных на уровне TCP.
- Освоены сложные фильтры отображения для удобного выделения конкретных адресов, сеансов, условий и типов пакетов (по флагам TCP или содержимому).
- Также удалось поработать со статистическим инструментарием Wireshark, включая анализ TCP-сессий и применение фильтров на основе выделенных соединений.

Wireshark очень эффективен при работе с сетями и пакетами, как инструмент для диагностики и мониторинга сетевых проблем, безопасности, а так же хорошо помогает обучаться основам администрирования и сетей. Вдобавок, данная программа позволяет успешно находить нужные айпишники, адреса и в дальнейшем при работе с другим железом перенастраивать конфигурацию для дальнейшего подключения. Так, к примеру, при работе с различными одноплатниками, сетевые интерфейсы могут быть сбиты или подключены не к тем сетям, однако Wireshark поможет проанализировать и исправить неполадки.

Особое внимание было уделено аспектам информационной безопасности: анализ незашифрованного трафика (SMTP, POP3, FTP) наглядно показал, насколько уязвимы такие протоколы к перехвату конфиденциальной информации, включая логины и пароли.

Полученные навыки являются фундаментальными для дальнейшей профессиональной деятельности в области сетевых технологий и кибербезопасности. Таким образом, мы успешно поработали с Wireshark, получили новые практически навыки при работе с пакетами и адресами. Поработали с захватом данных, а также получили чёткое представление о роли и предназначение сетевых протоколов, которые необходимы для обеспечения корректной и безопасной передачи данных.